

AaoStays Entity API Documentation

USER MANAGEMENT API DOCUMENTATION

Base URL:

<https://api.aaostays.com/v1>

Authentication:

Bearer Token required

Authorization: Bearer {your_access_token}

Overview:

The User API manages all platform users including guests, hosts, admins, and employees. It stores basic authentication and profile details.

This document contains all APIs required for real-world user management, authentication, verification, password recovery, profile control, and security.

Includes all external APIs used for Email OTP and SMS OTP.

External APIs Used for User Entity

Mailtrap Email API

Purpose: Email OTP, password reset email, signup verification, notifications, welcome emails

External Endpoint:

POST <https://send.api.mailtrap.io/api/send>

Used by:

Email OTP

Forgot Password

Password Reset

Signup Email Verification

Notification Emails

Twilio SMS API

Purpose: SMS OTP, mobile verification, login OTP, high-security operations

External Endpoint:

POST

Used by:

Mobile OTP

Login OTP

Phone Verification

Password Reset OTP (if using SMS)

1. Create User (Admin/System-Level)

Request Type: POST

Endpoint: /api/v1/users

Description:

Creates a new user record in the system. This is used by administrators or internal systems, not public signup.

Why This API Is Needed:

Allows admin to create Guest, Host, Admin, or Employee users

Supports internal registration workflows

Used for onboarding hosts or assigning employee accounts

How It Works:

Backend validates that the email is unique

Password is hashed using BCrypt

User record is saved to the database

Email OTP can be automatically triggered using Mailtrap (optional)

Request Body:

```
{
  "full_name": "John Doe",
  "email": "john@example.com",
  "phone_number": "+91-9876543210",
  "password": "SecurePass123",
  "user_type": "GUEST",
  "address": "Sector 62, Noida"
}
```

201 Created:

```
{
  "status": 201,
  "message": "User created successfully",
}
```

```
"data": {  
  "user_id": 101,  
  "email": "john@example.com",  
  "user_type": "GUEST",  
  "created_at": "2025-11-11T10:00:00Z"  
}
```

409 Conflict:

```
{  
  "status": 409,  
  "message": "Email is already registered"  
}
```

2. Get User by ID

Request Type: GET

Endpoint: /api/v1/users/{userId}

Description:

Retrieves detailed information of a user using their unique ID.

Why This API Is Needed:

Required for user profile pages

Used by admin dashboards

Helps validate user identity during support or bookings

How It Works:

Backend fetches the user from database

Returns user data except password

If ID does not exist → return 404

200 OK:

```
{  
  "status": 200,  
  "data": {  
    "user_id": 101,  
    "full_name": "John Doe",  
    "email": "john@example.com",  
    "phone_number": "+91-9876543210",
```

```
"user_type": "GUEST",  
"address": "Sector 62, Noida",  
"created_at": "2025-11-11T10:00:00Z"  
}  
}
```

404 Not Found:

```
{  
"status": 404,  
"message": "User not found with ID: 101"  
}
```

3. List All Users

Request Type: GET

Endpoint: /api/v1/users

Description:

Returns a list of users with optional filters for admin dashboards.

Why This API Is Needed:

Useful for platform analytics

Allows admin to filter users by type and registration date

Supports pagination for large user bases

How It Works:

Backend applies filters (role, date range)

Returns paginated user data

Query Params:

user_type: string

created_from: date

created_to: date

200 OK:

```
{  
"status": 200,  
"data": [  
{
```

```
"user_id": 101,  
"full_name": "John Doe",  
"email": "john@example.com",  
"user_type": "GUEST"  
},  
{  
  "user_id": 102,  
  "full_name": "Priya Sharma",  
  "email": "priya@example.com",  
  "user_type": "HOST"  
}  
],  
"meta": {  
  "total_users": 2,  
  "page": 1  
}  
}
```

4. Update User

Request Type: PUT

Endpoint: /api/v1/users/{userId}

Description:

Modifies user profile details.

Why This API Is Needed:

Users need to keep contact details updated for bookings

Admins may update incorrect user information

How It Works:

System finds user

Allowed fields are updated

Timestamp updated_at is refreshed

Request Body:

```
{  
  "full_name": "Johnathan Doe",  
  "phone_number": "+91-9988776655",  
}
```

```
"address": "Noida Extension"
}
```

200 OK:

```
{
  "status": 200,
  "message": "User updated successfully",
  "data": {
    "user_id": 101,
    "updated_at": "2025-11-11T12:00:00Z"
  }
}
```

404 Not Found:

```
{
  "status": 404,
  "message": "User does not exist"
}
```

5. Delete User

Request Type: DELETE

Endpoint: /api/v1/users/{userId}

Description:

Deactivates or removes a user from the platform.

Why This API Is Needed:

Required for GDPR and account removal

Helps remove fake, inactive, or unwanted accounts

How It Works:

User is marked inactive or deleted based on rules

200 OK:

```
{
  "status": 200,
  "message": "User deleted successfully"
}
```

6. User Login

Request Type: POST

Endpoint: /api/v1/auth/login

Description:

Authenticates user and generates JWT tokens.

How It Works:

Backend checks email and BCrypt password

Generates access token and refresh token

Logs the session

Request Body:

```
{  
  "email": "john@example.com",  
  "password": "SecurePass123"  
}
```

200 OK:

```
{  
  "status": 200,  
  "message": "Login successful",  
  "access_token": "jwt_token_here",  
  "refresh_token": "refresh_token_here"  
}
```

401 Unauthorized:

```
{  
  "status": 401,  
  "message": "Invalid email or password"  
}
```

7. Signup (Public User Registration)

Request Type: POST

Endpoint: /api/v1/auth/register

Description:

Registers a new user and triggers email and SMS verification.

Why Needed:

Public platform onboarding

Required before booking or hosting

8. Email OTP – Send OTP (Mailtrap Integration)

Request Type: POST

Endpoint: /api/v1/auth/email/send-otp

External API Used: Mailtrap

External Endpoint: <https://send.api.mailtrap.io/api/send>

How It Works:

Backend generates OTP

Backend calls Mailtrap API to send OTP

OTP stored temporarily for verification

200 OK:

```
{  
  "status": 200,  
  "message": "Email OTP sent successfully"  
}
```

9. Email OTP – Verify OTP

Request Type: POST

Endpoint: /api/v1/auth/email/verify

Description:

Verifies OTP sent to the user's email.

10. SMS OTP – Send OTP (Twilio Integration)

Request Type: POST

Endpoint: /api/v1/auth/sms/send-otp

External API Used: Twilio

External Endpoint:

How It Works:

Backend generates OTP

Calls Twilio API to send OTP via SMS

Saves OTP for verification

200 OK:

```
{  
  "status": 200,  
  "message": "SMS OTP sent successfully"  
}
```

11. SMS OTP – Verify OTP

Request Type: POST

Endpoint: /api/v1/auth/sms/verify

Description:

Validates OTP delivered through Twilio SMS.

12. Password Management – Forgot Password

Request Type: POST

Endpoint: /api/v1/auth/forgot-password

Uses Mailtrap to send a password reset OTP.

Mailtrap Payload Example:

```
{  
  "subject": "AaoStays Password Reset",  
  "text": "Use OTP 983211 to reset your password.",  
  "to": [{ "email": "john@example.com" }]  
}
```

13. Password Reset – Final Step

Request Type: POST

Endpoint: /api/v1/auth/reset-password

Description:

Resets the password after verifying OTP.

14. Change Password (Logged-In User)

Request Type: PATCH

Endpoint: /api/v1/users/{id}/change-password

Description:

Allows authenticated user to change their password.

GUEST MANAGEMENT API DOCUMENTATION

Base URL:

Authentication: Bearer Token required

Authorization: Bearer {your_access_token}

Overview:

The Guest Management API handles the creation, update, and verification of guest profiles. Each guest is linked to a User record and contains additional details required for booking, check-in, identity verification, and record management.

1. Create Guest

Request Type: POST

Endpoint: /api/v1/guests

Description:

Creates a new guest profile linked to an existing user account.

Why This API Is Needed:

Stores guest-specific information (ID proof, DOB, address)

Separates guest profile from the User entity

Required for bookings, legal purposes, and hotel check-in

How It Works:

System verifies that the guest email is unique

Validates that the user_id exists

Stores the guest record in the database

Request Body:

```
{  
  "user_id": 101,  
  "guest_full_name": "Riya Sharma",
```

```
"email": "riya@example.com",
"phone_number": "+91-9812345678",
"gender": "Female",
"date_of_birth": "1998-06-12",
"address": "Connaught Place, New Delhi",
"id_proof_type": "Aadhar",
"id_proof_number": "1234-5678-9101"
}
```

Success Response:

```
{
"status": "success",
"message": "Guest created successfully",
"data": {
"guest_id": 201,
"user_id": 101,
"email": "riya@example.com",
"created_at": "2025-11-11T10:20:00Z"
}
}
```

Conflict Response:

```
{
"status": "error",
"error": {
"code": "GUEST_EXISTS",
"message": "Guest with this email already exists"
}
}
```

2. Get Guest by ID

Request Type: GET

Endpoint: /api/v1/guests/{guest_id}

Description:

Fetches a guest's full details using their guest ID.

Why This API Is Needed:

Required for booking review, check-in, or profile display

Helps admin/staff verify guest identity

How It Works:

System retrieves the guest from the database

If not found, returns a 404 error

Success Response:

```
{
  "status": "success",
  "data": {
    "guest_id": 201,
    "user_id": 101,
    "guest_full_name": "Riya Sharma",
    "email": "riya@example.com",
    "phone_number": "+91-9812345678",
    "gender": "Female",
    "address": "Connaught Place, New Delhi",
    "id_proof_type": "Aadhar",
    "id_proof_number": "1234-5678-9101",
    "created_at": "2025-11-11T10:20:00Z"
  }
}
```

Error Response:

```
{
  "status": "error",
  "error": {
    "code": "GUEST_NOT_FOUND",
    "message": "Guest not found with ID 201"
  }
}
```

3. List All Guests

Request Type: GET

Endpoint: /api/v1/guests

Description:

Fetches a list of all guests with optional filters.

Why This API Is Needed:

Useful for admin dashboards

Helps easily search or filter guests

Supports customer support operations

Query Parameters:

email: string

date_from: date

date_to: date

Success Response:

```
{
  "status": "success",
  "data": [
    {
      "guest_id": 201,
      "guest_full_name": "Riya Sharma",
      "email": "riya@example.com"
    },
    {
      "guest_id": 202,
      "guest_full_name": "Amit Verma",
      "email": "amit@example.com"
    }
  ],
  "meta": {
    "total_guests": 2,
    "page": 1
  }
}
```

4. Update Guest

Request Type: PUT

Endpoint: /api/v1/guests/{guest_id}

Description:

Updates an existing guest's profile details.

Why This API Is Needed:

Allows guests to update personal information

Ensures accurate data for bookings and communication

Request Body:

```
{
  "phone_number": "+91-9812340000",
```

```
"address": "New Friends Colony, Delhi"
}
```

Success Response:

```
{
  "status": "success",
  "message": "Guest updated successfully",
  "data": {
    "guest_id": 201,
    "updated_at": "2025-11-11T13:00:00Z"
  }
}
```

5. Delete Guest

Request Type: DELETE

Endpoint: /api/v1/guests/{guest_id}

Description:

Deletes or deactivates a guest profile.

Why This API Is Needed:

Removes inactive or duplicate guests

Required for data cleanup or user-controlled deletion

Success Response:

```
{
  "status": "success",
  "message": "Operation successful"
}
```

6. Verify Guest Identity (Additional API 1.1)

Request Type: POST

Endpoint: /api/v1/guests/{guest_id}/verify-identity

Description:

Verifies guest ID information (Aadhar, PAN, Passport) before allowing bookings or check-in.

Why This API Is Needed:

Legally required by hotels

Prevents fake or fraudulent guests

Needed for secure check-in

How It Works:

Backend validates id_proof_type and id_proof_number

Optional integration with government ID verification services

Marks guest as verified

Success Response:

```
{  
  "status": "success",  
  "message": "Guest identity verified successfully",  
  "data": {  
    "guest_id": 201,  
    "verified": true  
  }  
}
```

7. Upload Guest Identity Documents (Additional API 1.2)

Request Type: POST

Endpoint: /api/v1/guests/{guest_id}/id-upload

Description:

Uploads the guest's identity proof documents (Aadhar, PAN, Passport).

Why This API Is Needed:

Required for digital check-in

Stores KYC documents securely

Reduces manual verification time at hotels

External API Used: Cloudinary (Images)

Cloudinary Upload Endpoint:

POST

How It Works:

Guest uploads ID document image

Backend sends image to Cloudinary

Cloudinary returns secure URL

URL is saved to the Guest record

Request Body Example:

Form-data with file upload: id_document_file

Success Response:

```
{  
  "status": "success",  
  "message": "ID document uploaded successfully",  
  "url": "https://res.cloudinary.com/.../guest_id_proof.jpg"  
}
```

8. Guest Booking Confirmation Email (External API 2.2 – Mailtrap)

Request Type: POST

Endpoint: /api/v1/guests/{guest_id}/send-confirmation-email

Description:

Sends a booking confirmation email to a guest using Mailtrap.

Why This API Is Needed:

Notifies guest about successful booking

Provides booking summary

Enhances communication and trust

External API Used: Mailtrap

Mailtrap Email Endpoint:

POST https://send.api.mailtrap.io/api/send

Backend Payload Example:

```
{  
  "to": [{ "email": "riya@example.com" }],  
  "subject": "Your AaoStays Booking Confirmation",  
  "text": "Your booking is confirmed. Thank you for choosing AaoStays!"  
}
```

Success Response:

```
{  
  "status": "success",  
  "message": "Booking confirmation email sent successfully"  
}
```


Final Summary of Included Guest APIs

Core Guest APIs:

Create Guest

Get Guest

List Guests

Update Guest

Delete Guest

Additional Internal APIs Included:

Verify Guest Identity

Upload Guest Identity Documents

External APIs Included:

Cloudinary (ID Document Upload)

Mailtrap (Guest Booking Emails)

HOST MANAGEMENT API DOCUMENTATION

Base URL:

Authentication: Bearer Token required

Authorization: Bearer {your_access_token}

Overview:

The Host API manages property owners who list, manage, and operate properties on AaoStays.

A host is linked to the User entity and includes business, legal, and verification details.

Additional internal and external APIs are required for KYC, compliance, property onboarding, and communication.

1. Create Host

Request Type: POST

Endpoint: /api/v1/hosts

Description:

Creates a new host profile linked to an existing user account.

Why This API Is Needed:

Registers a user as a Host

Stores company, legal, and government ID information

Needed before property listing and onboarding can begin

How It Works:

Backend verifies that user_id exists

Checks if host email already exists

Stores host and business information in Host table

Request Body:

```
{
  "user_id": 102,
  "host_full_name": "Priya Mehta",
  "email": "priya@example.com",
  "phone_number": "+91-9900011122",
  "government_id_type": "PAN",
  "government_id_number": "ABCDE1234F",
  "company_name": "Mehta Hospitality",
  "gst_number": "29ABCDE1234F1Z5",
  "address": "MG Road, Bengaluru"
}
```

Success Response (201 Created):

```
{
  "status": "success",
  "message": "Host account created successfully",
  "data": {
    "host_id": 301,
    "user_id": 102,
    "email": "priya@example.com",
    "company_name": "Mehta Hospitality",
    "created_at": "2025-11-11T14:20:00Z"
  }
}
```

2. Get Host by ID

Request Type: GET

Endpoint: /api/v1/hosts/{host_id}

Description:

Retrieves complete host profile details.

Why This API Is Needed:

Required for host dashboards

Used for admin verification before property approval

Displays business details and account status

How It Works:

Retrieves existing host record

If no host found, returns 404

Success Response:

```
{
  "status": "success",
  "data": {
    "host_id": 301,
    "host_full_name": "Priya Mehta",
    "email": "priya@example.com",
    "phone_number": "+91-9900011122",
    "company_name": "Mehta Hospitality",
    "gst_number": "29ABCDE1234F1Z5",
    "address": "MG Road, Bengaluru",
    "status": "ACTIVE",
    "created_at": "2025-11-11T14:20:00Z"
  }
}
```

3. List All Hosts

Request Type: GET

Endpoint: /api/v1/hosts

Description:

Returns host list with optional filters.

Why This API Is Needed:

Used in admin back office for managing hosts

Helps track host businesses and compliance status

Query Parameters:

status: ACTIVE / INACTIVE

city: string

company_name: string

Success Response:

```
{
  "status": "success",
  "data": [
    {
      "host_id": 301,
      "host_full_name": "Priya Mehta",
      "company_name": "Mehta Hospitality"
    },
    {
      "host_id": 302,
      "host_full_name": "Arjun Singh",
      "company_name": "Singh Stays"
    }
  ]
}
```

4. Update Host

Request Type: PUT

Endpoint: /api/v1/hosts/{host_id}

Description:

Updates business, identity, or contact details of a host.

Why This API Is Needed:

Hosts frequently update GST, ID proof, and business information

Ensures accurate compliance records

Updates host display information on the platform

Request Body:

```
{
  "company_name": "Mehta Luxury Stays",
```

```
"gst_number": "29ABCDE1234F1Z9",  
"address": "Indiranagar, Bengaluru"  
}
```

Success Response:

```
{  
  "status": "success",  
  "message": "Host updated successfully",  
  "data": {  
    "host_id": 301,  
    "updated_at": "2025-11-11T15:00:00Z"  
  }  
}
```

5. Delete Host

Request Type: DELETE

Endpoint: /api/v1/hosts/{host_id}

Description:

Removes or deactivates a host.

Why This API Is Needed:

For non-active hosts

Compliance, fraud control, or business closure

Success Response:

```
{  
  "status": "success",  
  "message": "Operation successful"  
}
```

ADDITIONAL INTERNAL APIs REQUIRED FOR HOST ENTITY

These APIs are mandatory for a real-world host management system.

6. Verify Host Identity (KYC Verification)

Request Type: POST

Endpoint: /api/v1/hosts/{host_id}/verify-identity

Description:

Verifies the host's government ID and business legitimacy.

Why This API Is Needed:

Required before approving property listings

Ensures commercial operators are legitimate

Prevents fraud and unauthorized property listings

How It Works:

Validates government_id_type and government_id_number

Optionally integrates with external legal verification APIs

Marks host as verified

Success Response:

```
{  
  "status": "success",  
  "message": "Host identity verified successfully",  
  "data": {  
    "host_id": 301,  
    "verified": true  
  }  
}
```

7. Upload Host Government ID Document

Request Type: POST

Endpoint: /api/v1/hosts/{host_id}/id-upload

Description:

Uploads scanned images of host identity documents (PAN, Passport, GST certificate).

Why This API Is Needed:

Mandatory for regulatory compliance

Required before property onboarding

Helps prevent fraudulent host accounts

External API Used: Cloudinary

Cloudinary Upload Endpoint:

POST

Request Body:

Form-data containing ID document file

Success Response:

```
{  
  "status": "success",  
  "message": "ID document uploaded successfully",  
  "url": "https://res.cloudinary.com/.../host_id_proof.jpg"  
}
```

8. Host Status Management

Request Type: PATCH

Endpoint: /api/v1/hosts/{host_id}/activate

Endpoint: /api/v1/hosts/{host_id}/deactivate

Description:

Activates or deactivates host account status.

Why This API Is Needed:

Compliance action for fraudulent or inactive hosts

Admin-level control to manage host participation

Allows temporary suspension

Success Response:

```
{  
  "status": "success",  
  "message": "Host status updated successfully"  
}
```

9. Host Property List (Internal)

Request Type: GET

Endpoint: /api/v1/hosts/{host_id}/properties

Description:

Retrieves all properties listed by the host.

Why This API Is Needed:

Required for host dashboard

Helps admin verify host activity

Needed to manage bookings and listings

Success Response:

```
{
  "status": "success",
  "data": [
    {
      "property_id": 501,
      "property_name": "Mehta Luxury Villa"
    }
  ]
}
```

ADDITIONAL EXTERNAL APIs REQUIRED FOR HOST ENTITY

These external systems support host onboarding and communication.

10. Cloudinary (Document and Business File Uploads)

Used For:

Government ID images

GST certificate images

Business documents

Why Needed:

Cloud storage

Secured file hosting

Quick retrieval for admin verification

11. Mailtrap or SendGrid (Host Notifications)

Used For:

Host welcome emails

Property approval notifications

Payment settlement updates

Mailtrap Endpoint:

POST <https://send.api.mailtrap.io/api/send>

Why Needed:

Keeps hosts informed

Handles business-critical communication

FINAL SUMMARY — Host Entity APIs Required

Core CRUD APIs:

Create Host

Get Host

List Hosts

Update Host

Delete Host

Internal Host APIs Added:

Verify Host Identity

Upload Government ID

Activate Host

Deactivate Host

Host Property List

External APIs Required:

Cloudinary (document storage)

Mailtrap/SendGrid (host notifications)

ADMIN MANAGEMENT API DOCUMENTATION

Base URL:

Authentication: Bearer Token required

Authorization: Bearer {your_access_token}

Overview:

The Admin API manages platform administrators responsible for system operations, approvals, verification workflows, and data governance.

Admins have role-based permissions and may require additional verification and activity tracking for security and compliance.

1. Create Admin

Request Type: POST

Endpoint: /api/v1/admins

Description:

Creates a new admin profile linked to an existing User account.

Why This API Is Needed:

Adds new admin or staff accounts to the system

Assigns roles such as SUPER_ADMIN, MANAGER, SUPPORT

Controls permissions for sensitive areas (verification, payouts, KYC, etc.)

How It Works:

Validates that the user_id exists

Ensures admin email is unique

Assigns permissions array

Stores admin roles securely

Request Body:

```
{
  "user_id": 1,
  "admin_full_name": "Rahul Kapoor",
  "email": "rahul.kapoor@aaostays.com",
  "phone_number": "+91-9990001112",
  "role": "SUPER_ADMIN",
  "permissions": ["MANAGE_USERS", "VERIFY_HOSTS", "VIEW_REPORTS"]
}
```

Success Response (201 Created):

```
{
  "status": "success",
  "message": "Admin created successfully",
  "data": {
    "admin_id": 501,
    "email": "rahul.kapoor@aaostays.com",
    "role": "SUPER_ADMIN",
    "created_at": "2025-11-11T14:50:00Z"
  }
}
```

Error Response:

```
{
  "status": "error",
  "message": "Error message details"
}
```

2. Get Admin by ID

Request Type: GET

Endpoint: /api/v1/admins/{admin_id}

Description:

Fetches detailed admin information using the admin ID.

Why This API Is Needed:

Required for admin dashboards

Allows system to view admin roles and access permissions

Used in audit and verification processes

Success Response:

```
{
  "status": "success",
  "data": {
    "admin_id": 501,
    "admin_full_name": "Rahul Kapoor",
    "email": "rahul.kapoor@aaostays.com",
    "role": "SUPER_ADMIN",
    "permissions": ["MANAGE_USERS", "VERIFY_HOSTS", "VIEW_REPORTS"],
    "created_at": "2025-11-11T14:50:00Z"
  }
}
```

```
}  
}
```

Error Response:

```
{  
  "status": "error",  
  "message": "Error message details"  
}
```

3. List All Admins

Request Type: GET

Endpoint: /api/v1/admins

Description:

Returns a list of all admins with role details.

Why This API Is Needed:

Helps manage admin hierarchy

Useful for compliance audits

Required for role monitoring and internal operations

Success Response:

```
{  
  "status": "success",  
  "data": [  
    { "admin_id": 501, "name": "Rahul Kapoor", "role": "SUPER_ADMIN" },  
    { "admin_id": 502, "name": "Sonia Verma", "role": "MANAGER" }  
  ]  
}
```

Error Response:

```
{  
  "status": "error",  
  "message": "Error message details"  
}
```

4. Update Admin

Request Type: PUT

Endpoint: /api/v1/admins/{admin_id}

Description:

Updates admin role or permissions.

Why This API Is Needed:

Roles change based on responsibilities

Access permissions must be updated for internal security

Helps manage admin hierarchy

Request Body:

```
{  
  "role": "MANAGER",  
  "permissions": ["MANAGE_USERS", "VIEW_REPORTS"]  
}
```

Success Response:

```
{  
  "status": "success",  
  "message": "Admin updated successfully"  
}
```

Error Response:

```
{  
  "status": "error",  
  "message": "Error message details"  
}
```

5. Delete Admin

Request Type: DELETE

Endpoint: /api/v1/admins/{admin_id}

Description:

Removes or deactivates an admin account.

Why This API Is Needed:

Required when admin leaves the organization

Needed for internal security and access removal

Helps prevent unauthorized access

Success Response:

```
{
```

```
"status": "success",  
"message": "Admin deleted successfully"  
}
```

Error Response:

```
{  
"status": "error",  
"message": "Error message details"  
}
```

ADDITIONAL INTERNAL ADMIN APIS REQUIRED FOR PRODUCTION

Below are required internal APIs for a complete admin management system.

6. Admin Login Audit Logs

Request Type: GET

Endpoint: /api/v1/admins/{admin_id}/audit-logs

Description:

Returns login and activity logs for an admin.

Why Needed:

Mandatory for internal security

Detect suspicious admin behavior

Audit trail for compliance

Success Response:

```
{  
"status": "success",  
"data": [  
  {  
    "timestamp": "2025-11-11T10:20:00Z",  
    "action": "LOGIN",  
    "ip_address": "122.176.19.21"  
  }  
]  
}
```

7. Admin Access Level Update

Request Type: PATCH

Endpoint: /api/v1/admins/{admin_id}/permissions

Description:

Updates admin permissions dynamically.

Why Needed:

Role-based access control (RBAC)

Prevents giving excessive permissions

Required for scalable admin management

Request Body:

```
{  
  "permissions": ["MANAGE_USERS", "VIEW_REPORTS"]  
}
```

8. Admin Password Reset (Internal Only)

Request Type: POST

Endpoint: /api/v1/admins/{admin_id}/reset-password

Description:

Allows SUPER_ADMIN to reset another admin's password.

Why Needed:

Critical for access recovery

Required when admin is locked out

Ensures controlled password reset

Request Body:

```
{  
  "new_password": "SecurePass123"  
}
```

9. Admin Status Management (Activate / Deactivate)

Request Type: PATCH

Endpoints:

/api/v1/admins/{admin_id}/activate

/api/v1/admins/{admin_id}/deactivate

Description:

Holds or restores admin access.

Why Needed:

Suspend admin accounts for misconduct

Temporary access removal

Internal control for high-risk admin accounts

NECESSARY EXTERNAL APIs FOR ADMIN ENTITY (PRODUCTION)

These APIs are required for messaging, verification, and audit processes.

10. Mailtrap or SendGrid (Admin Email Notifications)

Used For:

Admin welcome email

Security alerts

Password reset notifications

System updates

Mailtrap Email Endpoint:

POST <https://send.api.mailtrap.io/api/send>

Why Needed:

Ensures reliable internal email communication

Keeps admin aware of system events

11. Twilio or Fast2SMS (Admin Alerts)

Used For:

Security alerts

Login OTP for Admin (optional high-security)

Twilio Endpoint:
POST

12. Cloudinary (Admin Document Upload – Optional)

Used For:

Uploading admin verification documents (if required)

Uploading internal compliance documents

Cloudinary Upload Endpoint:
POST

FINAL SUMMARY OF REQUIRED ADMIN APIS

Core APIs:

Create Admin

Get Admin

List Admins

Update Admin

Delete Admin

Internal Production APIs:

Admin Login Audit Logs

Update Permissions

Reset Admin Password

Activate / Deactivate Admin

External APIs Needed:

Mailtrap / SendGrid (email)

Twilio / Fast2SMS (SMS security alerts)

Cloudinary (document storage, optional)

EMPLOYEE MANAGEMENT API DOCUMENTATION

Base URL:

Authentication: Bearer Token required

Authorization: Bearer {your_access_token}

Overview:

The Employee API manages internal AaoStays staff members such as support, operations, field agents, quality control employees, and corporate staff.

Employees are linked to the User Entity and require access control, verification, activity logging, and department-level management.

1. Create Employee

Request Type: POST

Endpoint: /api/v1/employees

Description:

Creates a new employee profile linked to an existing User record.

Why This API Is Needed:

Used for onboarding new internal staff

Assigns employee roles and department structure

Required for giving platform access and responsibilities

How It Works:

Verifies User exists

Ensures email is unique for employees

Saves internal employee metadata (department, designation, salary)

Request Body:

```
{  
  "user_id": 1101,  
  "employee_name": "Anita Joshi",  
  "email": "anita@aaostays.com",  
  "phone_number": "+91-8899001122",  
  "department": "Support",  
  "designation": "Customer Service Executive",
```

```
"salary": 45000
}
```

Success Response:

```
{
  "status": "success",
  "message": "Employee created successfully",
  "data": {
    "employee_id": 601,
    "employee_name": "Anita Joshi",
    "department": "Support"
  }
}
```

Error Response:

```
{
  "status": "error",
  "message": "Error message details"
}
```

2. Get Employee by ID

Request Type: GET

Endpoint: /api/v1/employees/{employee_id}

Description:

Fetches detailed employee information using employee_id.

Why This API Is Needed:

For internal HR dashboards

For verifying employee details during escalations

Required for tracking designation and department history

How It Works:

Retrieves employee profile

Excludes sensitive user_authentication fields

Success Response:

```
{
  "status": "success",
  "data": {
```

```
"employee_id": 601,  
"employee_name": "Anita Joshi",  
"email": "anita@aaostays.com",  
"department": "Support",  
"designation": "Customer Service Executive",  
"salary": 45000,  
"created_at": "2025-11-12T09:20:00Z"  
}  
}
```

Error Response:

```
{  
"status": "error",  
"message": "Employee not found"  
}
```

3. List All Employees

Request Type: GET

Endpoint: /api/v1/employees

Description:

Returns a list of all employees with optional filters.

Why This API Is Needed:

Helps HR/admins view and manage staff

Supports department-level management and analytics

Helps allocate employees to tasks and operations

Query Filters (optional):

department: string

designation: string

status: ACTIVE / INACTIVE

Success Response:

```
{  
"status": "success",  
"data": [  
  {  
    "employee_id": 601,  
    "employee_name": "Anita Joshi",  
    "department": "Support"
```

```
},  
{  
  "employee_id": 602,  
  "employee_name": "Ravi Kumar",  
  "department": "Operations"  
}  
]  
}
```

Error Response:

```
{  
  "status": "error",  
  "message": "Error message details"  
}
```

4. Update Employee

Request Type: PUT

Endpoint: /api/v1/employees/{employee_id}

Description:

Updates internal employee information such as designation or salary.

Why This API Is Needed:

Allows HR or Admins to promote, transfer, or update compensation

Keeps employee job information up to date

Necessary for payroll and reporting

Request Body:

```
{  
  "designation": "Senior Support Executive",  
  "salary": 52000  
}
```

Success Response:

```
{  
  "status": "success",  
  "message": "Employee updated successfully"  
}
```

Error Response:

```
{
```

```
"status": "error",  
"message": "Error message details"  
}
```

5. Delete Employee

Request Type: DELETE

Endpoint: /api/v1/employees/{employee_id}

Description:

Deactivates or removes an employee.

Why This API Is Needed:

Required when an employee leaves the organization

Needed for access removal and internal system security

Supports HR offboarding workflows

Success Response:

```
{  
"status": "success",  
"message": "Employee deleted successfully"  
}
```

Error Response:

```
{  
"status": "error",  
"message": "Error message details"  
}
```

ADDITIONAL INTERNAL EMPLOYEE APIS REQUIRED FOR PRODUCTION

These APIs are required for real HR management, security, and operations.

6. Employee Attendance / Check-in Tracking

Request Type: POST

Endpoint: /api/v1/employees/{employee_id}/attendance

Description:

Logs when an employee checks in/out for work.

Why Needed:

Required for HR attendance tracking

Supports payroll calculations

Offers operational visibility

Success Response:

```
{  
  "status": "success",  
  "message": "Attendance recorded successfully"  
}
```

7. Employee Status Management (Activate / Deactivate)

Request Type: PATCH

Endpoints:

/api/v1/employees/{employee_id}/activate

/api/v1/employees/{employee_id}/deactivate

Description:

Updates the activity status of an employee account.

Why Needed:

Prevents system access after resignation or suspension

Adds internal control on account lifecycle

Success Response:

```
{  
  "status": "success",  
  "message": "Employee status updated"  
}
```

8. Employee Role or Department Transfer

Request Type: PATCH

Endpoint: /api/v1/employees/{employee_id}/transfer

Description:

Transfers employee to a new department or changes their designation.

Why Needed:

Common HR workflow

Needed for growing organizations with internal movements

Request Body:

```
{  
  "new_department": "Quality Control",  
  "new_designation": "QC Executive"  
}
```

9. Employee Performance Notes (Internal Only)

Request Type: POST

Endpoint: /api/v1/employees/{employee_id}/notes

Description:

Stores HR/manager performance notes for employees.

Why Needed:

Used for appraisals

Helps track behaviour, warnings, achievements

Request Body:

```
{  
  "note": "Excellent performance during peak booking season."  
}
```

10. Employee Access Log

Request Type: GET

Endpoint: /api/v1/employees/{employee_id}/access-logs

Description:

Tracks employee login, logout, and access events.

Why Needed:

Security auditing

Insider threat prevention

Required for compliance

Success Response:

```
{
  "status": "success",
  "data": [
    {
      "timestamp": "2025-11-10T08:22:00Z",
      "action": "LOGIN",
      "ip": "122.155.220.13"
    }
  ]
}
```

EXTERNAL APIS REQUIRED FOR EMPLOYEE ENTITY (PRODUCTION)

These external integrations support internal communication, security, and document management.

11. Mailtrap or SendGrid (Employee Notifications)

Used for:

Employee onboarding email

Offboarding notification

Payroll notification (optional)

System alerts

Mailtrap Endpoint:

POST <https://send.api.mailtrap.io/api/send>

Why Needed:

Simplifies internal HR communication

Ensures reliable delivery

12. Twilio or Fast2SMS (Employee Alerts)

Used for:

Sending login alerts

Emergency notifications

Critical updates to field employees

Twilio SMS Endpoint:

POST

13. Cloudinary (Optional Employee Docs)

Used for:

Storing employee documents (ID copies, verification files)

Cloudinary Upload Endpoint:

POST

FINAL SUMMARY — EMPLOYEE ENTITY REQUIREMENTS

Core CRUD APIs:

Create Employee

Get Employee

List Employees

Update Employee

Delete Employee

Internal APIs Needed:

Employee Attendance

Employee Status Management

Department/Designation Transfer

Performance Notes

Access Logs

External APIs Needed:

Mailtrap/SendGrid (email)

Twilio/Fast2SMS (sms alerts)

Cloudinary (optional document storage)

AMENITY MANAGEMENT API DOCUMENTATION

Base URL:

Authentication: Bearer Token required

Authorization: Bearer {your_access_token}

Overview:

The Amenity API manages property or room amenities such as Wi-Fi, AC, Parking, Laundry, Housekeeping, etc.

Amenities are reusable master records that can be mapped to rooms or properties.

These APIs ensure clean amenity management and allow admins to update, categorize and maintain consistent options across the platform.

1. Create Amenity

Request Type: POST

Endpoint: /api/v1/amenities

Description:

Creates a new amenity in the system.

Why This API Is Needed:

Allows creation of reusable amenity definitions

Ensures consistency across property listings

Used by admins to configure available features

Request Body:

```
{  
  "amenity_name": "Free Wi-Fi",  
  "amenity_description": "High-speed internet access",  
  "amenity_type": "ROOM"  
}
```

Success Response:

```
{  
  "status": "success",  
  "message": "Amenity created successfully",  
  "data": {
```

```
"amenity_id": 101,  
"amenity_name": "Free Wi-Fi"  
}  
}
```

Error Response:

```
{  
"status": "error",  
"message": "Amenity already exists"  
}
```

2. Get Amenity by ID

Request Type: GET

Endpoint: /api/v1/amenities/{amenity_id}

Description:

Fetches detailed information about a specific amenity.

Why This API Is Needed:

Required in admin dashboards

Used by hosts while creating property listings

Success Response:

```
{  
"status": "success",  
"data": {  
"amenity_id": 101,  
"amenity_name": "Free Wi-Fi",  
"amenity_description": "High-speed internet access",  
"amenity_type": "ROOM",  
"status": "ACTIVE"  
}  
}
```

3. List All Amenities

Request Type: GET

Endpoint: /api/v1/amenities

Description:

Returns all amenities, optionally filtered.

Filters (optional):

amenity_type: ROOM / PROPERTY

status: ACTIVE / INACTIVE

Why This API Is Needed:

Used for filtering in property management

Helps admins maintain clean amenity categories

Success Response:

```
{
  "status": "success",
  "data": [
    { "amenity_id": 101, "amenity_name": "Free Wi-Fi" },
    { "amenity_id": 102, "amenity_name": "Air Conditioning" }
  ]
}
```

4. Update Amenity

Request Type: PUT

Endpoint: /api/v1/amenities/{amenity_id}

Description:

Updates amenity name, description, or type.

Why This API Is Needed:

For correcting or enhancing amenity details

Ensures up-to-date feature information

Request Body:

```
{
  "amenity_name": "Premium Wi-Fi",
  "amenity_description": "Ultra high-speed internet",
  "amenity_type": "ROOM"
}
```

Success Response:

```
{
  "status": "success",
  "message": "Amenity updated successfully"
}
```

5. Delete Amenity

Request Type: DELETE

Endpoint: /api/v1/amenities/{amenity_id}

Description:

Deletes or deactivates an amenity.

Why This API Is Needed:

Required when features become outdated

Helps keep listing options clean

Success Response:

```
{  
  "status": "success",  
  "message": "Amenity removed successfully"  
}
```

ADDITIONAL INTERNAL AMENITY APIS REQUIRED FOR PRODUCTION

These are essential for real-world system operations.

6. Activate / Deactivate Amenity

Request Type: PATCH

Endpoint: /api/v1/amenities/{amenity_id}/activate

Endpoint: /api/v1/amenities/{amenity_id}/deactivate

Description:

Used to enable or disable an amenity without deleting it.

Why Needed:

Prevents breaking existing rooms that use the amenity

Allows temporary disabling

Success Response:

```
{  
  "status": "success",
```

```
"message": "Amenity status updated"
}
```

7. Upload Amenity Icon (External Integration)

Request Type: POST

Endpoint: /api/v1/amenities/{amenity_id}/icon-upload

Description:

Uploads an icon/image representing the amenity.

External API Used: Cloudinary

Cloudinary Upload Endpoint:

POST

Why Needed:

Improves UI with visual amenity indicators

Used in property cards and booking pages

Success Response:

```
{
  "status": "success",
  "message": "Icon uploaded successfully",
  "icon_url": "https://res.cloudinary.com/.../wifi_icon.png"
}
```

8. Amenity Search API

Request Type: GET

Endpoint: /api/v1/amenities/search

Description:

Searches amenities by name or type.

Why Needed:

Useful for admin dashboards

Helps hosts quickly find amenity options

Query Example:

/amenities/search?query=wifi

Success Response:

```
{
  "status": "success",
  "data": [
    { "amenity_id": 101, "amenity_name": "Free Wi-Fi" }
  ]
}
```

ROOM AMENITY MANAGEMENT API DOCUMENTATION

Base URL:

Authentication: Bearer Token required

Authorization: Bearer {your_access_token}

Overview:

The Room Amenity API manages the mapping between specific rooms and amenities. This enables property owners to configure which amenities belong to each room.

1. Create Room Amenity

Request Type: POST

Endpoint: /api/v1/room-amenities

Description:

Creates a mapping between a room and an amenity.

Why This API Is Needed:

Required to assign amenities to rooms

Ensures flexible property configuration

Request Body:

```
{
  "room_id": 401,
  "amenity_id": 101,
  "is_available": true
}
```

Success Response:

```
{
  "status": "success",
  "message": "Room amenity created"
}
```


2. Get Room Amenity by ID

Request Type: GET

Endpoint: /api/v1/room-amenities/{room_amenity_id}

Description:

Retrieves room-amenity mapping details.

Why This API Is Needed:

Required for room detail pages

Used by admin panels

Success Response:

```
{
  "status": "success",
  "data": {
    "room_amenity_id": 501,
    "room_id": 401,
    "amenity_id": 101,
    "is_available": true
  }
}
```

3. List All Room Amenities

Request Type: GET

Endpoint: /api/v1/room-amenities

Description:

Returns all room-amenity mappings.

Filters:

room_id

amenity_id

is_available

Why This API Is Needed:

Provides full amenity details for rooms

Helps in property editing flows

Success Response:

```
{
  "status": "success",
  "data": [
    { "room_amenity_id": 501, "room_id": 401, "amenity_id": 101 }
  ]
}
```

4. Update Room Amenity

Request Type: PUT

Endpoint: /api/v1/room-amenities/{room_amenity_id}

Description:

Updates mapping fields such as availability or notes.

Why This API Is Needed:

Allows updating amenity availability

Helps manage temporary unavailability (e.g., under maintenance)

Request Body:

```
{
  "is_available": false,
  "notes": "Under maintenance"
}
```

Success Response:

```
{
  "status": "success",
  "message": "Room amenity updated successfully"
}
```

5. Delete Room Amenity

Request Type: DELETE

Endpoint: /api/v1/room-amenities/{room_amenity_id}

Description:

Removes a mapping between room and amenity.

Why This API Is Needed:

Used when a room no longer offers the amenity

Success Response:

```
{  
  "status": "success",  
  "message": "Room amenity removed"  
}
```

ADDITIONAL INTERNAL ROOM_AMENITY APIS REQUIRED

6. Bulk Assign Amenities to Room

Request Type: POST

Endpoint: /api/v1/room-amenities/bulk-assign

Description:

Assigns multiple amenities to a single room in one request.

Request Body:

```
{  
  "room_id": 401,  
  "amenity_ids": [101, 102, 103]  
}
```

7. Get Amenities by Room ID

Request Type: GET

Endpoint: /api/v1/rooms/{room_id}/amenities

Description:

Returns all amenities assigned to a room.

8. Room Amenity Availability Sync

Request Type: PATCH

Endpoint: /api/v1/room-amenities/{id}/toggle-availability

Description:

Toggles availability of an amenity for a room.

FINAL SUMMARY

Included Amenity APIs:

Create, Get, List, Update, Delete

Activate/Deactivate

Icon Upload (Cloudinary)

Search

Included Room Amenity APIs:

Create, Get, List, Update, Delete

Bulk Assign

Get by Room

Toggle Availability

External APIs Needed:

Cloudinary (icon and images)

PROPERTY MANAGEMENT API DOCUMENTATION

Base URL:

Authentication: Bearer Token required

Authorization: Bearer {your_access_token}

Overview

The Property API manages all property listings on AaoStays.

It allows hosts and admins to create, update, delete, search, and retrieve property details, including location, pricing, images, amenities, policies, and approval workflows.

Every property belongs to a host (host_id).

A property can contain rooms, images, amenities, policies, and availability details.

1. Add Property

Request Type: POST

Endpoint: /api/v1/properties

Controller Method: addProperty(PropertyDto propertyDto)

Description

Creates a new property listing. Only hosts or admins can create properties.

Why This API Is Needed

Adds a new listing into the platform

Required for hosts to begin bookings

Creates a unique property profile linked to a host

Initiates approval workflow (PENDING_APPROVAL)

How It Works

Validates required fields (name, address, hostId, etc.)

Saves details in PROPERTY table

Sets initial status: PENDING_APPROVAL

Returns propertyId

Request Body

```
{
  "hostId": 101,
  "propertyName": "The South Villa",
  "propertyType": "Entire Place",
  "categoryType": "Luxury",
  "description": "A stunning beachfront villa with private pool.",
  "addressLine1": "Beach Road, Candolim",
  "city": "Goa",
  "state": "Goa",
  "country": "India",
  "postalCode": "403515",
  "latitude": 15.5187,
  "longitude": 73.7596,
  "baseGuests": 2,
  "maxGuests": 4,
  "extraGuestAllowed": true,
  "extraGuestFee": 500.00,
  "bedrooms": 2,
  "beds": 3,
  "bathrooms": 2,
  "kitchenType": "Full Kitchen",
  "petsAllowed": false,
  "smokingAllowed": false,
```

```
"eventsAllowed": true,
"pricePerNight": 8500.00,
"cleaningFee": 500.00,
"serviceFee": 200.00,
"currency": "INR",
"minimumStay": 2,
"maximumStay": 30,
"checkInTime": "14:00:00",
"checkOutTime": "11:00:00",
"cancellationPolicy": "MODERATE",
"instantBooking": false,
"propertyStatus": "PENDING_APPROVAL",
"approvalStatus": "PENDING",
"isActive": true,
"isFeatured": false
}
```

Success Response (201 Created)

```
{
  "status": 201,
  "message": "Property created successfully",
  "data": {
    "propertyId": 501,
    "propertyName": "The South Villa",
    "propertyStatus": "PENDING_APPROVAL",
    "approvalStatus": "PENDING",
    "createdAt": "2025-11-10T14:12:45Z"
  }
}
```

Error Responses

400 Bad Request

Missing or invalid fields

```
{
  "status": 400,
  "message": "Validation failed: propertyName must not be blank"
}
```

401 Unauthorized

```
{
```

```
"status": 401,  
"message": "Unauthorized access. Please login to continue."  
}
```

403 Forbidden

```
{  
"status": 403,  
"message": "Access denied. Only hosts or admins can create properties."  
}
```

409 Conflict

```
{  
"status": 409,  
"message": "Property already exists with the same name and address."  
}
```

500 Internal Server Error

```
{  
"status": 500,  
"message": "An unexpected error occurred while saving property. Please try again later."  
}
```

503 Service Unavailable

```
{  
"status": 503,  
"message": "Service temporarily unavailable. Please try again later."  
}
```

2. Get All Properties

Request Type: GET

Endpoint: /api/v1/properties

Controller Method: getAllProperties()

Description

Returns a list of all active properties. Supports filtering.

Filters Supported

city, state, propertyType, categoryType, minPrice, maxPrice, rating, status

Success Response (200 OK)

```
{
  "status": 200,
  "message": "Properties retrieved successfully",
  "data": [
    {
      "propertyId": 501,
      "propertyName": "The South Villa",
      "city": "Goa",
      "pricePerNight": 8500.00,
      "currency": "INR",
      "propertyStatus": "ACTIVE",
      "ratingAverage": 4.9
    }
  ]
}
```

204 No Content

```
{
  "status": 204,
  "message": "No properties found"
}
```

500 Internal Server Error

```
{
  "status": 500,
  "message": "An unexpected error occurred while fetching properties."
}
```

3. Get Property by ID

Request Type: GET

Endpoint: /api/v1/properties/{propertyId}

Controller Method: getPropertyById(Long propertyId)

Description

Fetches detailed information for a single property.

Success Response (200 OK)

```
{
  "status": 200,
  "message": "Property details retrieved successfully",
  "data": {
    "propertyId": 501,
    "propertyName": "The South Villa",
    "propertyType": "Entire Place",
    "categoryType": "Luxury",
    "description": "A stunning beachfront villa with private pool.",
    "city": "Goa",
    "state": "Goa",
    "pricePerNight": 8500.00,
    "currency": "INR",
    "ratingAverage": 4.9,
    "approvalStatus": "APPROVED",
    "propertyStatus": "ACTIVE"
  }
}
```

404 Not Found

```
{
  "status": 404,
  "message": "Property not found with ID: 501"
}
```

400 Bad Request

```
{
  "status": 400,
  "message": "Invalid property ID provided"
}
```

4. Update Property

Request Type: PUT

Endpoint: /api/v1/properties/{propertyId}

Description

Updates existing property details. Only host or admin allowed.

Example Request

```
{
  "pricePerNight": 9000.00,
  "maxGuests": 5,
  "description": "Updated: Includes free breakfast."
}
```

Success Response (200 OK)

```
{
  "status": 200,
  "message": "Property updated successfully",
  "data": {
    "propertyId": 501,
    "propertyName": "The South Villa",
    "pricePerNight": 9000.00,
    "maxGuests": 5,
    "updatedAt": "2025-11-10T15:12:00Z"
  }
}
```

Error Responses

400 Invalid data

403 Unauthorized host

404 Property not found

500 Update operation failed

5. Delete Property

Request Type: DELETE

Endpoint: /api/v1/properties/{propertyId}

Description

Deletes or deactivates a property.

Success Response (200 OK)

```
{
  "status": 200,
  "message": "Property deleted successfully",
  "data": {
    "propertyId": 501,
    "deletedAt": "2025-11-10T15:30:00Z"
  }
}
```

Error Responses

401 Unauthorized

403 Forbidden (host mismatch)

404 Property not found

500 Server error

PROPERTY SEARCH AND FILTER APIs

6. Property Search

Request Type: GET

Endpoint: /api/v1/properties/search

Description:

Dynamic multi-filter search for properties.

Supports:

city, state, category, type, minPrice, maxPrice

Success Response (200 OK)

```
{
  "status": 200,
  "message": "Properties fetched successfully",
  "data": [...]
}
```

404 Not Found

```
{  
  "status": 404,  
  "message": "No properties found matching search criteria"  
}
```

7. Filter by City

GET /api/v1/properties/filter/city/{city}

Returns all properties in a specific city.

8. Filter by Price Range

GET /api/v1/properties/filter/price?minPrice=3000&maxPrice=8000

INTERNAL PRODUCTION APIS REQUIRED FOR PROPERTY MODULE

Below are essential internal APIs used by real booking platforms.

9. Update Property Status (Admin Only)

PATCH /api/v1/properties/{id}/status

Used to move property from PENDING_APPROVAL → APPROVED → ACTIVE → BLOCKED.

10. Update Approval Status (Admin Only)

PATCH /api/v1/properties/{id}/approval

Used for admin workflow and compliance.

11. Update Property Location

PATCH /api/v1/properties/{id}/location

Updates lat/lon using OpenStreetMap (Nominatim) reverse geocoding.

12. Add Property Policies

POST /api/v1/properties/{id}/policies

Stores cancellation, house rules, event permissions, etc.

13. Add/Update Property Amenities (Bulk)

POST /api/v1/properties/{id}/amenities

Assigns multiple amenities at once.

14. Add/Update Property Images (Cloudinary)

POST /api/v1/properties/{id}/images

Uploads files to Cloudinary → stores URL in PROPERTY_IMAGE table.

EXTERNAL API INTEGRATION FOR PROPERTY MODULE

A. Mapping API (OpenStreetMap + Nominatim)

Used For

Forward geocoding (address → lat/lon)

Reverse geocoding (lat/lon → address)

Map tile display on frontend

Endpoint Examples

/search?q=<address>&format=json

/reverse?lat=<lat>&lon=<lon>&format=json

B. Email API (Mailtrap / Gmail SMTP)

Used For

Property approval mail

Host notifications

Booking-related emails

C. SMS API (Twilio / Fast2SMS)

Used For

Instant SMS notifications to hosts

Booking confirmations

Property approval notifications

D. Image Upload API (Cloudinary)

Used For

Uploading property images

Optimizing and hosting images via CDN

Endpoint

POST

FINAL SUMMARY

Property Module Contains

Create, Read, Update, Delete

Search & Filters

Approval and workflow management

Location mapping

Image uploading

Amenities & policies linking

External APIs Used

OpenStreetMap (location)

Mailtrap/Gmail (email)

Twilio/Fast2SMS (sms)

Cloudinary (image hosting)

PROPERTY IMAGE MODULE API DOCUMENTATION

Base URL: /api/v1/properties

Entity: PROPERTY_IMAGE

Description:

The Property Image module manages all images uploaded for properties. Each property can have multiple images, including one primary (cover) image. This module handles uploading, sorting, updating, and deleting images associated with a property.

1. Internal APIs (Backend)

These APIs operate inside your system and are consumed by the frontend, admin, and mobile apps.

1.1 Add Property Image

Request Type: POST

Endpoint: /api/v1/properties/{propertyId}/images

Description:

Uploads or adds an image to a property. Image URLs typically come from Cloudinary.

Request Body

```
{  
  "imageUrl":  
  "https://res.cloudinary.com/aaostays/image/upload/v1731254321/villa1.jpg",  
  "imageCaption": "Ocean view from balcony",  
  "displayOrder": 1,  
  "isPrimary": true  
}
```

201 Created

```
{  
  "status": 201,  
  "message": "Property image added successfully",  
  "data": {  
    "imageId": 1001,  
    "propertyId": 501,  
    "imageUrl": "...",  
    "isPrimary": true  
  }  
}
```

400 Bad Request
Invalid or missing image URL.

404 Not Found
Property does not exist.

500 Internal Server Error

1.2 Get All Images for a Property

Request Type: GET
Endpoint: /api/v1/properties/{propertyId}/images

200 OK
Returns list of all images.

404 Not Found
No images found.

1.3 Get Primary Image

Request Type: GET
Endpoint: /api/v1/properties/{propertyId}/images/primary**

200 OK
Returns only the primary image.

404 Not Found
No primary image is marked.

1.4 Update Image Details

Request Type: PUT
Endpoint: /api/v1/properties/images/{imageId}**

Updates caption, display order, or primary flag.

1.5 Delete Property Image

Request Type: DELETE
Endpoint: /api/v1/properties/images/{imageId}**

Deletes a single image record.

1.6 Set Primary Image

Request Type: PATCH

Endpoint: /api/v1/properties/images/{imageId}/setPrimary**

Automatically unmarks other images as primary.

1.7 Delete All Images for a Property

Request Type: DELETE

Endpoint: /api/v1/properties/{propertyId}/images**

Deletes all images associated with a property.

1.8 Reorder Images

Request Type: PUT

Endpoint: /api/v1/properties/{propertyId}/images/reorder**

Reorders gallery images.

2. External APIs Required

The Property Image module must use Cloudinary for image upload, optimization, CDN delivery, and deletion.

2.1 Cloudinary Image Upload API

Endpoint:

Purpose:

Uploads property images from the backend or directly from the frontend.

Used For:

Uploading photos

Storing URLs

Auto-optimization

Transformations

Global CDN delivery

2.2 Cloudinary Image Destroy API

Endpoint:

https://api.cloudinary.com/v1_1/{cloud_name}/image/destroy

Purpose:

Deletes unused or outdated images from storage.

3. Internal Helper Services

These are backend services used internally (not endpoints).

File Validation Service

Checks file size, format, and URL validity.

Cloudinary Integration Service

Handles upload, delete, signature creation, and error handling.

4. Summary

The Property Image module is complete with:

Eight internal backend APIs

Two external Cloudinary APIs

Complete CRUD and ordering features

Production-ready architecture

No other APIs are required.

CONTACT DETAILS MODULE API DOCUMENTATION

Base URL: /api/v1/contact-details

Entity: CONTACT_DETAILS

Description:

Stores and manages contact information for each property, including email, phone numbers, and preferred communication method. Each property has exactly one contact details record.

1. Internal APIs (Backend)

1.1 Add Contact Details

Request Type: POST

Endpoint: /api/v1/contact-details/{propertyId}

Adds a new contact record (one per property).

1.2 Get Contact Details by Property ID

Request Type: GET

Endpoint: /api/v1/contact-details/{propertyId}

Returns name, email, phone numbers, and preferred method.

1.3 Update Contact Details

Request Type: PUT

Endpoint: /api/v1/contact-details/{propertyId}

Updates full contact details.

1.4 Delete Contact Details

Request Type: DELETE

Endpoint: /api/v1/contact-details/{propertyId}

Soft or hard deletion based on system rules.

1.5 Update Contact Email

Request Type: PATCH

Endpoint: /api/v1/properties/{propertyId}/contact/email**

Updates only the email field.

1.6 Update Contact Phone Numbers

Request Type: PATCH

Endpoint: /api/v1/properties/{propertyId}/contact/phone**

Updates one or both phone numbers.

1.7 Update Preferred Contact Method

Request Type: PATCH

Endpoint: /api/v1/properties/{propertyId}/contact/preferred-method**

Updates preferred method (PHONE, EMAIL, WHATSAPP).

2. External APIs Required

The Contact module may use external services for notifications.

2.1 Mailtrap / Gmail SMTP – Email Notification API

Base URLs:

smtp.mailtrap.io

smtp.gmail.com

Purpose:

Sends emails when:

Contact details are created

Contact details are updated

Email field is changed

Alerts are required

2.2 Twilio / Fast2SMS - SMS Notification API

Base URLs:

Twilio:

Fast2SMS:

Purpose:

Sends SMS when:

Phone numbers are updated

Verification is needed

Urgent communication is required

3. Internal Helper Services

Validation Service

Verifies email format, phone number format, contact method rules.

Notification Service

Triggers email and SMS when contact data changes.

4. Summary

The Contact Details module is complete with:

Seven internal endpoints

Two external notification integrations

Validation and notification helper services

Production-ready architecture

No additional APIs are needed.

PROPERTY CATEGORY API DOCUMENTATION

Base URL: /api/v1/property-categories

Entity: PROPERTY_CATEGORY

Description:

Manages the relationship between properties and categories. Categories help classify

properties by type or theme (Luxury, Beachfront, Family Stay, Budget, etc.). This is a many-to-many mapping where a single property can belong to multiple categories.

1. Internal APIs (Backend)

1.1 Add Property Category Link

Request Type: POST

Endpoint: /api/v1/property-categories

Description:

Links a property with a category.

Request Body:

```
{  
  
  "propertyId": 501,  
  
  "categoryId": 10  
  
}
```

201 Created

Link created successfully.

400 Bad Request

Missing propertyId or categoryId.

404 Not Found

Either property or category not found.

409 Conflict

Already linked.

500 Internal Server Error

1.2 Get Categories by Property ID

Request Type: GET

Endpoint: /api/v1/property-categories/property/{propertyId}**

Returns all assigned categories.

404 Not Found

If no categories assigned.

1.3 Get Properties by Category ID

Request Type: GET

Endpoint: /api/v1/property-categories/category/{categoryId}**

Returns list of properties for a category.

1.4 Delete Property Category Link

Request Type: DELETE

Endpoint: /api/v1/property-categories/{propertyCategoryId}**

Removes a single mapping.

1.5 Remove Category from Property

Request Type: DELETE

Endpoint: /api/v1/property-categories/property/{propertyId}/category/{categoryId}**

Removes by using both IDs.

1.6 Bulk Assign Categories to Property

Request Type: POST

Endpoint: /api/v1/properties/{propertyId}/categories**

Assigns multiple categories at once.

Request Body:

```
{  
  
  "categoryIds": [1, 3, 5]  
}
```

1.7 Remove All Categories from a Property

Request Type: DELETE

Endpoint: /api/v1/properties/{propertyId}/categories**

Deletes all category links for a property.

2. External APIs Required

None.

Property Category module is fully internal. No SMS, email, or external API is required.

3. Internal Helper Services

Validation Service

Ensures categoryId and propertyId are valid and unique per mapping.

Mapping Service

Handles bulk insert, conflict detection, and cleanup.

4. Summary

The Property Category module is complete with seven internal APIs. No external API is required because category assignment is a purely internal classification system. This module is production-ready.

PROPERTY AMENITY API DOCUMENTATION

Base URL: /api/v1/property-amenities

Entity: PROPERTY_AMENITY

Description:

Manages the many-to-many relationship between properties and amenities. Each property can have multiple amenities, with flags for availability and notes.

1. Internal APIs (Backend)

1.1 Add Property Amenity

Request Type: POST

Endpoint: /api/v1/property-amenities**

Creates a new property-amenity link.

Request Body:

```
{  
  "propertyId": 501,  
  "amenityId": 20,  
  "isAvailable": true,  
  "notes": "Available in all rooms"  
}
```

201 Created

Link created.

400 Bad Request

Missing propertyId or amenityId.

404 Not Found

Property or amenity not found.

409 Conflict

Already assigned.

500 Internal Server Error

1.2 Get Amenities by Property ID

Request Type: GET

Endpoint: /api/v1/property-amenities/property/{propertyId}**

Returns all amenities associated with a property.

404 Not Found

If no amenities found.

1.3 Get Properties by Amenity ID

Request Type: GET

Endpoint: /api/v1/property-amenities/amenity/{amenityId}**

Returns all properties that offer the amenity.

1.4 Update Amenity Availability or Notes

Request Type: PUT

Endpoint: /api/v1/property-amenities/{propertyAmenityId}**

Updates availability or notes.

1.5 Delete Property Amenity Link

Request Type: DELETE

Endpoint: /api/v1/property-amenities/{propertyAmenityId}**

Removes link.

1.6 Get Missing Amenities for a Property

Request Type: GET

Endpoint: /api/v1/property-amenities/property/{propertyId}/missing**

Returns amenities not yet assigned.

404 Not Found

If property invalid.

204 No Content

All amenities already assigned.

1.7 Bulk Assign Amenities

Request Type: POST

Endpoint: /api/v1/property-amenities/property/{propertyId}/bulk**

Assigns multiple amenities in a single request.

Request Body:

{

```
"amenities": [  
  { "amenityId": 20, "isAvailable": true },  
  { "amenityId": 23, "isAvailable": true }  
]  
}
```

1.8 Bulk Update Amenities

Request Type: PUT

Endpoint: /api/v1/property-amenities/property/{propertyId}/bulk-update**

Updates multiple assigned amenities.

2. External APIs Required

None.

Amenity assignment is fully internal.

No Cloudinary, SMTP, SMS, or map API is required.

3. Internal Helper Services

Validation Service

Ensures amenityId, propertyId, isAvailable, notes format.

Bulk Processor Service

Handles large assignment operations efficiently.

4. Summary

The Property Amenity module is complete with eight internal APIs handling assignment, update, deletion, missing amenities, and bulk operations.

No external integrations are required because amenities are metadata stored internally in the platform's relational database.

Final Note

Both:

Property Category Module

Property Amenity Module

are purely internal systems and do not require external APIs.

ROOM API DOCUMENTATION

Base URL: /api/v1/rooms

Entity: ROOM

Description:

The Room API manages all rooms under a property. Each room can have unique features, pricing, capacity, availability status, and other attributes. A property may contain multiple rooms.

This module is essential for room-level management, booking availability checks, host dashboards, and search filtering.

1. Internal APIs (Backend)

These are backend endpoints consumed by the app, admin panel, and booking system.

1.1 Add Room

Request Type: POST

Endpoint: /api/v1/rooms/{propertyId}

Description:

Creates a new room inside a property. Only property owners (hosts) and admins can add rooms.

Request Body

{

"roomNumber": "A101",

"roomType": "Deluxe",

"roomName": "Ocean View Suite",

```
"roomDescription": "Spacious suite with sea view, balcony, and modern amenities.",
"baseGuests": 2,
"maxGuests": 4,
"extraGuestAllowed": true,
"extraGuestFee": 500.00,
"pricePerNight": 8000.00,
"bedType": "King",
"bedCount": 1,
"roomSizeSqft": 350,
"hasBalcony": true,
"hasWindow": true,
"floorNumber": 1
}
```

201 Created

```
{
  "status": 201,
  "message": "Room added successfully",
  "data": {
    "roomId": 101,
    "propertyId": 501,
    "roomNumber": "A101",
    "roomType": "Deluxe",
    "roomStatus": "AVAILABLE",
    "pricePerNight": 8000.00,
    "createdAt": "2025-11-10T18:10:00Z"
  }
}
```

```
}
```

400 Bad Request
Validation failure.

404 Not Found
Property does not exist.

500 Internal Server Error

1.2 Get All Rooms by Property ID

Request Type: GET
Endpoint: /api/v1/rooms/property/{propertyId}
Description:
Fetches all rooms for a given property.

200 OK

```
{  
  "status": 200,  
  "message": "Rooms retrieved successfully",  
  "data": [  
    {  
      "roomId": 101,  
      "roomName": "Ocean View Suite",  
      "roomType": "Deluxe",  
      "pricePerNight": 8000.00,  
      "roomStatus": "AVAILABLE"  
    },  
    {  
      "roomId": 102,  
      "roomName": "Standard Room",  
      "roomType": "Standard",
```

```
"pricePerNight": 4500.00,  
"roomStatus": "OCCUPIED"  
}  
]  
}
```

404 Not Found

No rooms exist for this property.

500 Internal Server Error

1.3 Get Available Rooms by Property ID

Request Type: GET

Endpoint: /api/v1/rooms/property/{propertyId}/available

Description:

Returns only rooms with status AVAILABLE.

Used by booking engine and frontend room listing pages.

200 OK

Returns list of available rooms.

404 Not Found

No available rooms.

500 Internal Server Error

1.4 Search and Filter Rooms

Request Type: GET

Endpoint: /api/v1/rooms/search

Description:

Supports multi-parameter filtering of rooms by:

Room type

Price range

Status (AVAILABLE, OCCUPIED, MAINTENANCE)

Query Parameters

200 OK

Returns filtered list of rooms.

204 No Content

No rooms matched filter criteria.

500 Internal Server Error

1.5 Get Room by ID

Request Type: GET

Endpoint: /api/v1/rooms/{roomId}

Description:

Returns full room details including capacity, pricing, and features.

200 OK

Room details returned.

404 Not Found

Room ID does not exist.

1.6 Update Room Details

Request Type: PUT

Endpoint: /api/v1/rooms/{roomId}

Description:

Updates editable fields such as pricing, capacity, features, etc.

Request Body

```
{  
  "pricePerNight": 8500.00,  
  "roomStatus": "MAINTENANCE",  
  "maxGuests": 3  
}
```

200 OK

```
{  
  "status": 200,
```



```
"message": "Room updated successfully",  
"data": {  
  "roomId": 101,  
  "pricePerNight": 8500.00,  
  "roomStatus": "MAINTENANCE",  
  "updatedAt": "2025-11-10T18:15:00Z"  
}  
}
```

404 Not Found

500 Internal Server Error

1.7 Update Room Status

Request Type: PATCH

Endpoint: /api/v1/rooms/{roomId}/status

Description:

Updates room availability (AVAILABLE, BLOCKED, OCCUPIED, MAINTENANCE).

Request Body

```
{  
  "roomStatus": "BLOCKED"  
}
```

200 OK

Status updated.

404 Not Found

1.8 Delete Room

Request Type: DELETE

Endpoint: /api/v1/rooms/{roomId}

Description:

Deletes or disables a room depending on soft delete policy.

200 OK

Room deleted.

404 Not Found

500 Internal Server Error

1.9 Get Property Room Price Range

Request Type: GET

Endpoint: /api/v1/rooms/property/{propertyId}/price-range

Description:

Returns minimum and maximum room prices of a property. Used for filters and booking displays.

200 OK

Returns minPrice and maxPrice.

404 Not Found

500 Internal Server Error

2. External APIs Required

The ROOM module does not require any external APIs.

No SMS

No Email

No Cloudinary

No Map API

All operations are entirely internal through your room, property, and booking database tables.

3. Internal Helper Services

Room Validation Service

Validates:

Room number uniqueness

Guest capacity rules

Pricing rules

Room Pricing Engine

Used internally to calculate price range, discounts, and seasonal logic (optional).

Room Availability Service

Used internally to determine room status based on bookings (optional but recommended).

4. Summary

The Room Module is complete with:

Nine internal production-grade APIs

No external dependencies

Full CRUD for rooms

Search and filter capability

Status update

Price range operations

Host and admin dashboard compatibility

--

ROOM IMAGE API DOCUMENTATION

Base URL: /api/v1/rooms

Entity: ROOM_IMAGE

Authentication: Bearer Token required

Authorization: Host or Admin (for create/update/delete), Public read for GET endpoints
(subject to business rules)

Overview

The Room Image API manages images associated with individual rooms. Each room may have multiple images, one primary image, an ordered gallery sequence, captions, and optional metadata. Images are uploaded to an external image service (Cloudinary) and the returned URLs are stored in the ROOM_IMAGE table.

1. Core Endpoints

1.1 Add Room Image

Request Type: POST

Endpoint: /api/v1/rooms/{roomId}/images

Description

Adds a new image to a specific room. Typical flow: frontend uploads binary to Cloudinary (or backend proxies upload), receives secure URL, then calls this endpoint to register the image with the room.

Why This API Is Needed

Associates uploaded images with specific rooms.

Stores metadata (caption, order, primary) for gallery display.

Separates storage (Cloudinary) from application metadata.

How It Works

Frontend uploads file to Cloudinary (recommended) and gets an imageUrl.

Client submits imageUrl and metadata to this endpoint.

Backend validates roomId, imageUrl, and metadata.

If isPrimary=true, backend unmarks any other primary image for that room.

Backend saves record in ROOM_IMAGE table and returns created resource.

Request Body

```
{  
  "imageUrl":  
    "https://res.cloudinary.com/aaostays/image/upload/v1731254321/room1.jpg",  
  "imageCaption": "Balcony view",  
  "displayOrder": 1,  
  "isPrimary": true  
}
```

Success Response (201 Created)

```
{
```

```
"status": 201,  
"message": "Room image added successfully",  
"data": {  
  "imageId": 501,  
  "roomId": 101,  
  "imageUrl":  
"https://res.cloudinary.com/aaostays/image/upload/v1731254321/room1.jpg",  
  "isPrimary": true,  
  "displayOrder": 1,  
  "createdAt": "2025-11-10T19:00:00Z"  
}  
}
```

Error Responses

400 Bad Request — invalid payload or missing imageUrl.
404 Not Found — room not found with provided roomId.
409 Conflict — duplicate imageUrl for same room (optional check).
500 Internal Server Error — DB or service error.

1.2 Get All Images for a Room

Request Type: GET

Endpoint: /api/v1/rooms/{roomId}/images

Description

Retrieves all images linked to the specified room, ordered by displayOrder ascending.

Why This API Is Needed

Used by frontend gallery, room detail pages, and host dashboards.

How It Works

Backend queries ROOM_IMAGE by roomId, orders by displayOrder, returns list.

Success Response (200 OK)

```
{
```

```
"status": 200,  
"message": "Room images retrieved successfully",  
"data": [  
  {  
    "imageId": 501,  
    "imageUrl": "...",  
    "imageCaption": "Balcony view",  
    "isPrimary": true,  
    "displayOrder": 1  
  },  
  {  
    "imageId": 502,  
    "imageUrl": "...",  
    "imageCaption": "Living area",  
    "isPrimary": false,  
    "displayOrder": 2  
  }  
]  
}
```

Error Responses

404 Not Found — room does not exist or no images found (business choice: return 200 with empty list or 404).

500 Internal Server Error.

1.3 Get Primary Image for a Room

Request Type: GET

Endpoint: /api/v1/rooms/{roomId}/images/primary

Description

Returns the single primary image for the room if set.

Success Response (200 OK)

```
{  
  "status": 200,  
  "message": "Primary image retrieved successfully",  
  "data": {  
    "imageId": 501,  
    "imageUrl": "...",  
    "isPrimary": true  
  }  
}
```

Error Responses

404 Not Found — primary image not set.

500 Internal Server Error.

1.4 Update Room Image Details

Request Type: PUT

Endpoint: /api/v1/rooms/images/{imageId}

Description

Updates metadata for a room image: imageCaption, displayOrder, isPrimary.

How It Works

Validate imageId and payload.

If isPrimary=true, unmark previous primary for that room.

Save changes and return updated object.

Request Body

```
{  
  "imageCaption": "Updated caption",
```

```
"displayOrder": 2,  
"isPrimary": false  
}
```

Success Response (200 OK)

```
{  
  "status": 200,  
  "message": "Room image updated successfully",  
  "data": {  
    "imageId": 501,  
    "imageCaption": "Updated caption",  
    "displayOrder": 2,  
    "isPrimary": false,  
    "updatedAt": "2025-11-10T19:10:00Z"  
  }  
}
```

Error Responses

400 Bad Request — invalid fields.

404 Not Found — imageId not found.

500 Internal Server Error.

1.5 Delete Room Image

Request Type: DELETE

Endpoint: /api/v1/rooms/images/{imageId}

Description

Deletes a single room image and optionally triggers deletion on Cloudinary (recommended for cleanup).

How It Works

Validate imageId.

Remove DB record (soft or hard).

If configured, call Cloudinary destroy API to remove media asset.

If deleted image was primary, backend should optionally auto-promote another image.

Success Response (200 OK)

```
{  
  "status": 200,  
  "message": "Room image deleted successfully"  
}
```

Error Responses

404 Not Found — image not found.

500 Internal Server Error.

1.6 Set Primary Image

Request Type: PATCH

Endpoint: /api/v1/rooms/images/{imageId}/setPrimary

Description

Marks the specified image as primary for its room and unmarks any existing primary image.

Request Body (optional)

```
{  
  "isPrimary": true  
}
```

Success Response (200 OK)

```
{  
  "status": 200,  
  "message": "Image set as primary successfully",  
  "data": {  
    "imageId": 501,  
    "roomId": 101,
```

```
"isPrimary": true
}
}
```

Error Responses

400 Bad Request — invalid request.

404 Not Found — image not found.

500 Internal Server Error.

1.7 Reorder Room Images (Bulk Update)

Request Type: PUT

Endpoint: /api/v1/rooms/{roomId}/images/reorder

Description

Updates displayOrder for multiple images in a single request.

Request Body

```
{
  "imageOrder": [
    { "imageId": 501, "displayOrder": 1 },
    { "imageId": 502, "displayOrder": 2 }
  ]
}
```

Success Response (200 OK)

```
{
  "status": 200,
  "message": "Image order updated successfully",
  "updatedCount": 2
}
```

Error Responses

400 Bad Request — duplicate or invalid orders.

404 Not Found — missing image or room.

500 Internal Server Error.

1.8 Bulk Upload Room Images

Request Type: POST

Endpoint: /api/v1/rooms/{roomId}/images/bulk

Description

Accepts multiple imageUrls & metadata to assign several images at once (useful after multi-file Cloudinary upload).

Request Body

```
{
  "images": [
    { "imageUrl": "...", "imageCaption": "1", "displayOrder": 1, "isPrimary": true },
    { "imageUrl": "...", "imageCaption": "2", "displayOrder": 2, "isPrimary": false }
  ]
}
```

Success Response (201 Created)

```
{
  "status": 201,
  "message": "3 images uploaded and linked successfully",
  "data": { "createdCount": 3 }
}
```

Error Responses

400 Invalid list.

409 Duplicate images.

500 Internal Server Error.

1.9 Delete All Images for a Room

Request Type: DELETE

Endpoint: /api/v1/rooms/{roomId}/images

Description

Deletes every image linked to the room. Useful when removing or resetting a room.

Success Response (200 OK)

```
{  
  "status": 200,  
  "message": "All images deleted successfully for roomId 101",  
  "deletedCount": 5  
}
```

Error Responses

404 Room not found or no images.

500 Internal Server Error.

2. External APIs Required

2.1 Cloudinary — Image Upload API

Endpoint:

Purpose

Uploads binary images, returns secure urls for storage.

Performs transformations, optimizations, and CDN delivery.

Notes

Frontend may upload directly to Cloudinary using unsigned upload presets and then call backend endpoint with resulting URL.

Alternatively, backend may proxy uploads to keep credentials secure.

2.2 Cloudinary — Image Destroy API

Endpoint: https://api.cloudinary.com/v1_1/{cloud_name}/image/destroy

Purpose

Delete image assets when they are removed from the system to save storage and ensure compliance.

Final Summary

The Room Image module provides complete CRUD, bulk, and ordering operations for room-level media. It relies on Cloudinary for storage and delivery and includes helper services for validation, integration, processing, and cleanup. Endpoints support host/admin workflows and frontend gallery display; security best practices ensure safe handling of external URLs and sensitive credentials.

}

COUPON API DOCUMENTATION

Base URL:

/api/v1/coupons

Entity:

COUPON

Description:

Manages discount coupons that can be applied during bookings.

Coupons can be percentage-based or fixed-amount discounts and may include rules like validity dates, usage limits, and eligible user types.

1. Add Coupon

Request Type: POST

Endpoint: /api/v1/coupons

Description:

Creates a new coupon. Only administrators can create coupons.

Request Body

{

"couponCode": "AAO2025",

"couponName": "AaoStays Launch Offer",

```
"description": "Flat 20% discount for new users on first booking",
"discountType": "PERCENTAGE",
"discountValue": 20.00,
"maxDiscountAmount": 1000.00,
"minBookingAmount": 2000.00,
"validFrom": "2025-11-01",
"validUntil": "2025-12-31",
"usageLimit": 500,
"usageLimitPerUser": 1,
"applicableTo": "FIRST_BOOKING",
"applicableUserType": "NEW_USER",
"isActive": true,
"createdByAdminId": 1
}
```

201 Created

```
{
  "status": 201,
  "message": "Coupon created successfully",
  "data": {
    "couponId": 101,
    "couponCode": "AAO2025",
    "discountType": "PERCENTAGE",
    "discountValue": 20.00,
    "validFrom": "2025-11-01",
    "validUntil": "2025-12-31",
```

```
"isActive": true
}
}
400 Bad Request
{
  "status": 400,
  "message": "Validation failed: discountValue must be greater than 0"
}
409 Conflict
{
  "status": 409,
  "message": "Coupon code already exists"
}
500 Internal Server Error
{
  "status": 500,
  "message": "Error occurred while creating coupon"
}
```

2. Get All Coupons

Request Type: GET
Endpoint: /api/v1/coupons

Description:
Retrieves all coupons, including active and inactive.

200 OK

```
{
```

```
"status": 200,
"message": "Coupons retrieved successfully",
"data": [
  {
    "couponId": 101,
    "couponCode": "AAO2025",
    "couponName": "AaoStays Launch Offer",
    "discountType": "PERCENTAGE",
    "discountValue": 20.00,
    "validUntil": "2025-12-31",
    "isActive": true
  },
  {
    "couponId": 102,
    "couponCode": "WELCOME500",
    "couponName": "Welcome Bonus",
    "discountType": "FIXED_AMOUNT",
    "discountValue": 500.00,
    "validUntil": "2025-11-30",
    "isActive": false
  }
]
```

404 Not Found

```
{
  "status": 404,
```



```
"message": "No coupons found"
}
```

500 Internal Server Error

```
{
  "status": 500,
  "message": "Error occurred while fetching coupons"
}
```

3. Get Coupon by Code

Request Type: GET

Endpoint: /api/v1/coupons/{couponCode}

200 OK

```
{
  "status": 200,
  "message": "Coupon details retrieved successfully",
  "data": {
    "couponId": 101,
    "couponCode": "AAO2025",
    "couponName": "AaoStays Launch Offer",
    "discountType": "PERCENTAGE",
    "discountValue": 20.00,
    "maxDiscountAmount": 1000.00,
    "minBookingAmount": 2000.00,
    "validFrom": "2025-11-01",
    "validUntil": "2025-12-31",
  }
}
```

```
"usageLimit": 500,
"usedCount": 100,
"applicableUserType": "NEW_USER",
"isActive": true
}
}
404 Not Found
{
  "status": 404,
  "message": "Coupon not found with code: AAO2025"
}
```

4. Update Coupon

Request Type: PUT

Endpoint: /api/v1/coupons/{couponId}

Request Body

```
{
  "couponName": "Updated Launch Offer",
  "discountValue": 25.00,
  "validUntil": "2026-01-15",
  "isActive": true
}
```

200 OK

```
{
  "status": 200,
```

```
"message": "Coupon updated successfully",
```

```
"data": {
```

```
  "couponId": 101,
```

```
  "couponCode": "AAO2025",
```

```
  "discountValue": 25.00,
```

```
  "validUntil": "2026-01-15",
```

```
  "updatedAt": "2025-11-10T18:45:00Z"
```

```
}
```

```
}
```

404 Not Found

```
{
```

```
  "status": 404,
```

```
  "message": "Coupon not found with ID: 101"
```

```
}
```

500 Internal Server Error

```
{
```

```
  "status": 500,
```

```
  "message": "Error occurred while updating coupon"
```

```
}
```

5. Deactivate Coupon

Request Type: PATCH

Endpoint: /api/v1/coupons/{couponId}/deactivate

200 OK

```
{
```

```
"status": 200,  
"message": "Coupon deactivated successfully",  
"data": {  
  "couponId": 101,  
  "couponCode": "AAO2025",  
  "isActive": false  
}  
}
```

404 Not Found

```
{  
  "status": 404,  
  "message": "Coupon not found with ID: 101"  
}
```

6. Delete Coupon

Request Type: DELETE

Endpoint: /api/v1/coupons/{couponId}

200 OK

```
{  
  "status": 200,  
  "message": "Coupon deleted successfully"  
}
```

404 Not Found

```
{  
  "status": 404,
```

```
"message": "Coupon not found with ID: 101"
}

500 Internal Server Error

{
  "status": 500,
  "message": "Error occurred while deleting coupon"
}
```

7. Validate Coupon (Apply During Booking)

Request Type: POST

Endpoint: /api/v1/coupons/validate

Description:

Validates if a coupon is applicable for a booking based on booking amount, user type, eligibility rules, validity period, and usage limits.

Request Body

```
{
  "couponCode": "AAO2025",
  "userId": 2001,
  "bookingAmount": 5000.00
}
```

200 OK

```
{
  "status": 200,
  "message": "Coupon applied successfully",
  "data": {
```

```
"couponCode": "AAO2025",  
"discountAmount": 1000.00,  
"finalAmount": 4000.00  
}  
}  
400 Bad Request  
{  
  "status": 400,  
  "message": "Coupon is not valid for this booking or has expired"  
}
```

```
404 Not Found  
{  
  "status": 404,  
  "message": "Coupon not found or inactive"  
}
```

```
500 Internal Server Error  
{  
  "status": 500,  
  "message": "Error occurred while validating coupon"  
}
```

External APIs Used for Coupon System (If Needed)

Most coupon operations are internal. However, two optional external services can be used:

1. Email Notification (Mailtrap / Gmail SMTP)

Used to notify users when a coupon is awarded or redeemed.

2. Analytics / Events Tracking (Optional)

Mixpanel / GA4 can track coupon usage events:

coupon_viewed

coupon_applied

coupon_failed

(Not mandatory for MVP.)

NOTIFICATION API DOCUMENTATION

Base URL:

/api/v1/notifications

Entity:

NOTIFICATION

Description:

Manages user notifications including in-app, email, SMS, and push notifications.

Each notification is linked to a specific user and may correspond to a booking, payment, refund, property update, coupon, or system event.

1. Create Notification

Request Type: POST

Endpoint: /api/v1/notifications

Description:

Creates a notification and sends it to the user through the selected channels (IN_APP, EMAIL, SMS, PUSH).

Request Body

```
{  
  "userId": 1001,  
  "notificationType": "BOOKING_CONFIRMED",  
  "notificationChannel": "IN_APP",
```

```
"title": "Booking Confirmed!",  
"message": "Your booking #BKG12345 has been confirmed successfully.",  
"actionUrl": "https://aaostays.com/my-bookings/BKG12345",  
"relatedEntityType": "BOOKING",  
"relatedEntityId": 501,  
"priority": "NORMAL"  
}
```

201 Created

```
{  
  "status": 201,  
  "message": "Notification created successfully",  
  "data": {  
    "notificationId": 2001,  
    "userId": 1001,  
    "title": "Booking Confirmed!",  
    "notificationType": "BOOKING_CONFIRMED",  
    "isRead": false,  
    "createdAt": "2025-11-10T19:05:00Z"  
  }  
}
```

400 Bad Request

```
{  
  "status": 400,  
  "message": "Validation failed: userId and message are required"  
}
```


404 Not Found

```
{  
  "status": 404,  
  "message": "User not found with ID: 1001"  
}
```

500 Internal Server Error

```
{  
  "status": 500,  
  "message": "Error occurred while sending notification"  
}
```

2. Get All Notifications by User

Request Type: GET

Endpoint: /api/v1/notifications/user/{userId}

Description:

Retrieves all notifications for a user, with optional filters (e.g., read/unread).

Example Query Parameters

/api/v1/notifications/user/1001?isRead=false

200 OK

```
{  
  "status": 200,  
  "message": "Notifications retrieved successfully",  
  "data": [  
    {  
      "notificationId": 2001,
```

```
"title": "Booking Confirmed!",
"message": "Your booking #BKG12345 has been confirmed successfully.",
"isRead": false,
"notificationType": "BOOKING_CONFIRMED",
"priority": "NORMAL",
"createdAt": "2025-11-10T19:05:00Z"
},
{
  "notificationId": 2002,
  "title": "Payment Received",
  "message": "Your payment of ₹8500 has been successfully processed.",
  "isRead": true,
  "notificationType": "PAYMENT_SUCCESS",
  "priority": "NORMAL",
  "createdAt": "2025-11-09T11:32:00Z"
}
]
```

404 Not Found

```
{
  "status": 404,
  "message": "No notifications found for user ID: 1001"
}
```

500 Internal Server Error

```
{
  "status": 500,
```

```
"message": "Error occurred while fetching user notifications"
}
```

3. Get Notification by ID

Request Type: GET

Endpoint: /api/v1/notifications/{notificationId}

200 OK

```
{
  "status": 200,
  "message": "Notification details retrieved successfully",
  "data": {
    "notificationId": 2001,
    "userId": 1001,
    "title": "Booking Confirmed!",
    "message": "Your booking #BKG12345 has been confirmed successfully.",
    "notificationType": "BOOKING_CONFIRMED",
    "relatedEntityType": "BOOKING",
    "relatedEntityId": 501,
    "isRead": false,
    "priority": "NORMAL",
    "createdAt": "2025-11-10T19:05:00Z"
  }
}
```

404 Not Found

```
{
```

```
"status": 404,  
"message": "Notification not found with ID: 2001"  
}
```

4. Mark Notification as Read

Request Type: PATCH

Endpoint: /api/v1/notifications/{notificationId}/read

200 OK

```
{  
  "status": 200,  
  "message": "Notification marked as read successfully",  
  "data": {  
    "notificationId": 2001,  
    "isRead": true,  
    "readAt": "2025-11-10T19:15:00Z"  
  }  
}
```

404 Not Found

```
{  
  "status": 404,  
  "message": "Notification not found with ID: 2001"  
}
```

5. Mark All Notifications as Read (for a User)

Request Type: PATCH

Endpoint: /api/v1/notifications/user/{userId}/read-all

200 OK

```
{  
  "status": 200,  
  "message": "All notifications marked as read for user ID: 1001"  
}
```

404 Not Found

```
{  
  "status": 404,  
  "message": "No unread notifications found for user ID: 1001"  
}
```

6. Delete Notification

Request Type: DELETE

Endpoint: /api/v1/notifications/{notificationId}

200 OK

```
{  
  "status": 200,  
  "message": "Notification deleted successfully"  
}
```

404 Not Found

```
{  
  "status": 404,  
  "message": "Notification not found with ID: 2001"  
}
```

500 Internal Server Error

```
{  
  "status": 500,  
  "message": "Error occurred while deleting notification"  
}
```

7. Filter Notifications by Type or Priority

Request Type: GET

Endpoint: /api/v1/notifications/user/{userId}/filter

Description:

Filters notifications using query parameters such as notificationType and priority.

Example Query

/api/v1/notifications/user/1001/filter?type=PAYMENT_SUCCESS&priority=HIGH

200 OK

```
{  
  "status": 200,  
  "message": "Filtered notifications retrieved successfully",  
  "data": [  
    {  
      "notificationId": 2020,  
      "title": "Payment Success",  
      "message": "Your payment of ₹12,000 was successful.",  
      "priority": "HIGH",  
      "isRead": false  
    }  
  ]  
}
```

}

2. External APIs Required for Notifications

Below is the final and complete list of all external APIs needed to send real notifications beyond the app (email, SMS, push).

2.1 Mailtrap Email API (Transactional Emails) – REQUIRED

Purpose: Send booking, payment, refund, and verification emails.

Use Cases:

Booking Confirmed

Payment Received

Coupon Applied

Refund Processed

Welcome Email

Security Alerts

Password Reset

Type: SMTP + API

Base URL: smtp.mailtrap.io

2.2 Twilio SMS API (SMS + OTP) – REQUIRED

Purpose: Send SMS OTPs and important alerts.

Use Cases:

OTP Login / Signup

Booking Confirmed (SMS)

Payment Success / Failure

Host Alerts

Check-in Reminders

Guest Updates

Type: REST API

Base URL:

<https://api.twilio.com/2010-04-01/Accounts/{AccountSID}/Messages.json>

2.3 Firebase Cloud Messaging (FCM) – OPTIONAL (only if mobile app exists)

Purpose: Send push notifications to Android/iOS devices.

Use Cases:

Booking updates

Offers & promotions

Host alerts

Real-time payment status

App reminders

Type: HTTP v1

Base URL:

<https://fcm.googleapis.com/v1/projects/{projectId}/messages:send>

★ ♦ 3. Internal Helper Services (Mandatory for Production)

(Not external APIs — internal backend modules)

These make the notification system scalable and reliable.

3.1 Notification Dispatcher Service

Routes notifications:

In-App

Email (Mailtrap)

SMS (Twilio)

Push (FCM)

3.2 Template Engine Service

Handles HTML email templates and SMS text templates.

3.3 Queue Service (Recommended)

To avoid delays:

RabbitMQ / Kafka / AWS SQS

3.4 Notification Preference Manager

User settings for:

Email ON/OFF

SMS ON/OFF

Push ON/OFF

Stored under user_notification_settings.

✦ FINAL SUMMARY (Clean & Ready to Paste Into Word)

Notification External APIs Required:

Mailtrap Email API – for transactional emails

Twilio SMS API – for OTP + SMS alerts

Firebase Cloud Messaging (Optional) – for push notifications

Notification Internal Services Required:

Notification Dispatcher Service

Template Engine

Queue Worker (recommended)

Notification Preference Manager

Notification Internal APIs:

Create, fetch, filter, update, delete notifications (in-app)

User

Guest

Host

Admin

Employee

Amenity

Property

PROPERTY_IMAGE

CONTACT_DETAILS

PROPERTY_CATEGORY

PROPERTY_AMENITY

DISCOUNT

ROOM

Room_Image

Room_Amenities

Coupon

Booking

Booking_Guest

Payment

Review

Wishlist

Notification

Discount Management API Documentation

Base URL:

Authentication:

Authorization: Bearer {your_access_token}

Why Discount Management Is Needed

Discounts help increase bookings, improve conversion rates, attract new guests, reward loyal customers, and support seasonal or promotional campaigns. They enable property owners to implement flexible pricing strategies that improve occupancy and maximize revenue.

How Discount Management Works

Discounts are created by property managers or admins with rules such as type, percentage/amount, conditions, and validity periods.

Each discount is stored per property and classified as Early Bird, Weekly, Monthly, Seasonal, or Custom.

During booking, the pricing engine automatically checks booking dates, stay duration, user eligibility, and discount rules.

Only valid and active discounts matching the stay conditions are applied.

Admins can update, deactivate, or delete discounts at any time.

Bulk creation allows fast onboarding of seasonal or default discount sets.

Endpoints

1. Create Discount

Endpoint: POST /discounts

Description:

Creates a new discount rule for a property. Fields include type, value, validity, and eligibility conditions.

Use Case:

Used to add seasonal offers, promotional campaigns, long-stay discounts, or early-bird deals.

2. Get Discount by ID

Endpoint: GET /discounts/{discount_id}

Description:

Fetches full details of a discount using its unique ID.

Use Case:

Used when viewing or verifying discount configurations.

3. Get All Discounts for a Property

Endpoint: GET /properties/{property_id}/discounts

Description:

Lists all discounts assigned to a property (active, inactive, expired).

Use Case:

Used for property dashboards, reporting, and discount overview.

4. Get Applicable Discounts

Endpoint: POST /discounts/calculate

Description:

Returns eligible discounts for a booking based on stay duration, booking date, and property rules.

Use Case:

Used during checkout flow to calculate the best available discount.

5. Update Discount

Endpoint: PUT /discounts/{discount_id}

Description:

Replaces all fields of an existing discount. Allows editing values, conditions, and validity dates.

Use Case:

Used when modifying discount structure or extending offers.

6. Partially Update Discount

Endpoint: PATCH /discounts/{discount_id}

Description:

Updates only specific fields, such as toggling active/inactive status.

Use Case:

Used for temporary suspension or quick adjustments.

7. Delete Discount

Endpoint: DELETE /discounts/{discount_id}

Description:

Permanently removes a discount. Cannot delete if linked to active bookings.

Use Case:

Used to remove obsolete or replaced discounts.

8. Bulk Create Discounts

Endpoint: POST /properties/{property_id}/discounts/bulk

Description:

Creates multiple discounts at once for a property.

Use Case:

Used when onboarding a new property or launching seasonal discount campaigns.

BOOKING MANAGEMENT API DOCUMENTATION

Base URL

Authentication

All endpoints require Bearer token authentication:

Authorization: Bearer {your_access_token}

1. Overview

The Booking Entity manages the complete reservation process including property selection, pricing, payment, confirmation, refunds, cancellations, check-in/out, and statistics.

This module is central to the AaoStays system, controlling revenue, availability, and guest lifecycle.

2. Why Booking Entity Is Needed

Ensures real-time property availability and prevents double bookings

Supports dynamic pricing, coupons, discounts, and taxes

Tracks booking status (Pending, Confirmed, Checked-in, Cancelled...)

Handles payments & refunds securely

Records guest stay history for insights & loyalty

Connects to notifications (email/SMS), invoices, and property calendars

Without this module, the platform cannot operate as a booking marketplace.

3. How Booking Works (High Level)

Guest selects property & dates

System checks availability

Pricing engine calculates cost

Discount/coupon engine applies offers

Razorpay payment order created

Guest completes payment

Booking becomes CONFIRMED

Notifications sent (email + SMS)

Host manages check-in → check-out

Refunds/cancellations processed if required

4. Internal Services Used

4.1 Pricing Calculation Service

Computes base price, extra guest fees, taxes, discounts, and total.

4.2 Availability Engine

Verifies property availability & prevents overlapping bookings.

4.3 Coupon/Discount Engine

Validates coupon codes + calculates discount.

4.4 Payment Integration Service

Integrates with Razorpay for order creation & payment capture.

4.5 Notification Service

Sends booking confirmation, cancellation, refund alerts via email/SMS.

4.6 Refund Engine

Calculates refund according to cancellation policy.

5. External Services Required

5.1 Razorpay Payment Gateway (REQUIRED)

Used for:

Creating payment orders

Capturing payment

Issuing refunds

Fetching payment status

Sample Razorpay Order Create Payload

```
{  
  "amount": 238850,  
  "currency": "INR",
```

```
"receipt": "BKG-2025-98765-XKJM",  
"notes": {  
  "bookingId": "98765",  
  "userId": "5001"  
}  
}
```

5.2 Mailtrap Email API

Used for:

Booking confirmation email

Invoice email

Cancellation notice

Refund confirmation

5.3 Twilio SMS API

Used for sending SMS updates:

“Your booking is confirmed”

“Reminder: Check-in tomorrow”

Cancellation / refund alerts

6. BOOKING API ENDPOINTS

6.1 Create Booking

Endpoint: POST /bookings

Creates a new booking record.

Request Body


```
{  
  "property_id": 12345,  
  "room_id": 678,  
  "guest_id": 5001,  
  "coupon_id": 200,  
  "check_in_date": "2025-12-20",  
  "check_out_date": "2025-12-27",  
  "number_of_guests": 4,  
  "number_of_adults": 2,  
  "number_of_children": 2,  
  "number_of_infants": 0,  
  "special_requests": "Late check-in requested. Need high chair."  
}
```

Success Response (201)

```
{  
  "status": "success",  
  "data": {  
    "booking_id": 98765,  
    "booking_reference": "BK-2025-98765-XKJM",  
    "total_amount": 2388.50,  
    "booking_status": "PENDING",  
    "payment_status": "PENDING"  
  },  
  "message": "Booking created successfully"  
}
```

6.2 Get Booking by ID

Endpoint: GET /bookings/{booking_id}

Success Response

```
{
  "status": "success",
  "data": {
    "booking_id": 98765,
    "booking_reference": "BK-2025-98765-XKJM",
    "property_name": "Sunset Beach Villa",
    "room_name": "Ocean View Suite",
    "guest_name": "John Doe",
    "total_amount": 2388.50,
    "booking_status": "CONFIRMED"
  }
}
```

6.3 Get Booking by Reference

Endpoint: GET /bookings/reference/{booking_reference}

6.4 List Bookings (Filtering & Pagination)

Endpoint: GET /bookings

Supports powerful filters.

Example Response

```
{
  "status": "success",
  "data": [
```

```
{
  "booking_id": 98765,
  "property_name": "Sunset Beach Villa",
  "check_in_date": "2025-12-20",
  "booking_status": "CONFIRMED"
},
"meta": {
  "current_page": 1,
  "total_pages": 8
}
```

6.5 Update Booking

Endpoint: PUT /bookings/{booking_id}

Request Body

```
{
  "check_in_date": "2025-12-21",
  "check_out_date": "2025-12-28",
  "number_of_guests": 3,
  "special_requests": "Updated: Late check-in 10 PM"
}
```

6.6 Confirm Booking

Endpoint: POST /bookings/{booking_id}/confirm

This step typically confirms payment after Razorpay order paid webhook.

Request Body

```
{  
  "payment_method": "razorpay",  
  "transaction_id": "pay_LbA1Qp9sDXY123"  
}
```

6.7 Cancel Booking

Endpoint: POST /bookings/{booking_id}/cancel

Request Body

```
{  
  "cancellation_reason": "Guest changed plans",  
  "refund_requested": true  
}
```

6.8 Get Booking Price Breakdown

Endpoint: POST /bookings/calculate-price

Request Body

```
{  
  "property_id": 12345,  
  "room_id": 678,  
  "check_in_date": "2025-12-20",  
  "check_out_date": "2025-12-27",  
  "number_of_guests": 4,  
  "coupon_code": "WINTER20"  
}
```

Success Response

```
{
  "status": "success",
  "data": {
    "number_of_nights": 7,
    "pricing_breakdown": {
      "base_price": 2100.00,
      "discount_amount": 210.00,
      "tax_amount": 194.25,
      "total_amount": 2388.50
    }
  }
}
```

6.9 Get Property Availability

Endpoint: GET /properties/{property_id}/availability

6.10 Get Guest Bookings

Endpoint: GET /guests/{guest_id}/bookings

6.11 Process Refund

Endpoint: POST /bookings/{booking_id}/refund

Request Body

```
{
  "refund_amount": 2150.00,
  "refund_method": "original_payment_method",
  "refund_reason": "Free cancellation period"
```

```
}
```

6.12 Send Booking Confirmation Email

Endpoint: POST /bookings/{booking_id}/send-confirmation
Triggers Mailtrap email + optional PDF invoice.

6.13 Get Booking Statistics

Endpoint: GET /bookings/statistics

7. Razorpay Integration Summary

Order Creation

Before confirming booking:

```
{  
  "amount": 238850,  
  "currency": "INR",  
  "receipt": "BK-2025-98765",  
  "notes": {  
    "bookingId": 98765  
  }  
}
```

Refund API Payload

```
{  
  "amount": 215000,  
  "speed": "normal",  
  "receipt": "REFUND-98765"  
}
```

8. Notification Integrations

Mailtrap Email Example

```
{  
  "to": "john@example.com",  
  "subject": "Your Booking is Confirmed!",  
  "template": "booking-confirmation",  
  "variables": {  
    "guest_name": "John",  
    "booking_reference": "BK-2025-98765-XKJM"  
  }  
}
```

Twilio SMS Example

```
{  
  "to": "+918889991111",  
  "body": "Your booking BK-2025-98765-XKJM is confirmed. Check-in: 20 Dec."  
}
```

BOOKING_GUEST API Documentation

Base URL:

/api/booking-guests

Primary Table: BOOKING_GUEST
Primary Key: booking_guest_id
Foreign Key: booking_id → BOOKING(booking_id)

Overview

The Booking Guest API manages individual guest details associated with a booking. It allows the system to store and retrieve detailed information about every guest staying with the main booker — including name, contact details, age, and identification details. This is essential for hotel check-in, compliance, KYC verification, occupancy tracking, safety, and guest analytics.

Why This API Is Needed

Hotels and stays must record all guests, not just the primary booker.

Government regulations require ID proof for each staying guest.

Enables accurate occupancy tracking and guest verification at check-in.

Supports multi-guest stays (families, groups, corporate bookings).

Required for guest management, security, and reporting.

How It Works

A booking is created in the BOOKING table.

For each additional guest, a separate record is created in BOOKING_GUEST.

Properties can fetch all guests associated with a booking at check-in.

Guests can be updated or removed when plans change.

1. Create a Guest

Endpoint:
POST /api/booking-guests

Description:
Creates a new guest entry linked to an existing booking.

Request Body (JSON):

```
{  
  "booking_id": 1001,  
  "guest_full_name": "John Doe",  
  "guest_email": "john.doe@example.com",  
  "guest_phone": "+1-555-1234",  
  "guest_age": 32,  
  "guest_id_proof_type": "Passport",  
  "guest_id_proof_number": "P12345678",  
  "relationship_to_booker": "Colleague"  
}
```

Success Response:

```
{  
  "booking_guest_id": 501,  
  "booking_id": 1001,  
  "guest_full_name": "John Doe",  
  "guest_email": "john.doe@example.com",  
  "guest_phone": "+1-555-1234",  
  "guest_age": 32,  
  "guest_id_proof_type": "Passport",  
  "guest_id_proof_number": "P12345678",  
  "relationship_to_booker": "Colleague",  
  "created_at": "2025-11-10T18:25:43Z"  
}
```

Status Code: 201 Created

2. Retrieve All Guests for a Booking

Endpoint:

GET /api/booking-guests?booking_id={booking_id}

Description:

Returns all guests linked to a single booking.

Example Request:

GET /api/booking-guests?booking_id=1001

Example Response:

```
[
  {
    "booking_guest_id": 501,
    "booking_id": 1001,
    "guest_full_name": "John Doe",
    "guest_email": "john.doe@example.com",
    "guest_phone": "+1-555-1234",
    "guest_age": 32,
    "guest_id_proof_type": "Passport",
    "guest_id_proof_number": "P12345678",
    "relationship_to_booker": "Colleague",
    "created_at": "2025-11-10T18:25:43Z"
  },
  {
    "booking_guest_id": 502,
    "booking_id": 1001,
    "guest_full_name": "Jane Doe",
    "guest_email": "jane.doe@example.com",
    "guest_phone": "+1-555-5678",
```

```
"guest_age": 29,  
"guest_id_proof_type": "Aadhar",  
"guest_id_proof_number": "1234-5678-9012",  
"relationship_to_booker": "Spouse",  
"created_at": "2025-11-10T18:26:12Z"  
}  
]
```

Status Code: 200 OK

3. Retrieve a Single Guest

Endpoint:

GET /api/booking-guests/{booking_guest_id}

Description:

Fetches a specific guest's details.

Example Response:

```
{  
  "booking_guest_id": 501,  
  "booking_id": 1001,  
  "guest_full_name": "John Doe",  
  "guest_email": "john.doe@example.com",  
  "guest_phone": "+1-555-1234",  
  "guest_age": 32,  
  "guest_id_proof_type": "Passport",  
  "guest_id_proof_number": "P12345678",  
  "relationship_to_booker": "Colleague",  
  "created_at": "2025-11-10T18:25:43Z"  
}
```

Status Code: 200 OK

4. Update Guest Information

Endpoint:

PUT /api/booking-guests/{booking_guest_id}

Description:

Updates only the fields provided in the body.

Example Request:

```
{  
  "guest_phone": "+1-555-9999",  
  "guest_age": 33  
}
```

Response:

```
{  
  "message": "Guest information updated successfully."  
}
```

Status Code: 200 OK

5. Delete Guest Record

Endpoint:

DELETE /api/booking-guests/{booking_guest_id}

Description:

Deletes a guest entry permanently.

Example Response:

```
{  
  "message": "Guest deleted successfully."  
}
```

Status Code: 200 OK

Error Responses

Database Relationship

One booking → multiple booking guests

Deleting a booking removes all its guests (ON DELETE CASCADE)

Used for compliance, guest logs, and property check-in

PAYMENT API DOCUMENTATION

Base URL:

Authentication:

Bearer Token required

Authorization: Bearer {your_access_token}

Overview

The Payment API handles all money-related operations for bookings: creating payment records, initiating and verifying online payments via Razorpay, recording cash/other payments, processing refunds, and exposing endpoints for reconciliation and reporting. Payments are linked to bookings and guests. Every transaction is persisted in the PAYMENT table for audit, reporting, and refunds.

Why This API Is Needed

Payments are central to the booking lifecycle: confirm bookings, secure reservations, process refunds, and reconcile revenue. A dedicated Payments API ensures consistent transaction records, supports multiple payment methods (UPI, cards, wallets, net-banking, cash), and integrates securely with payment gateways (Razorpay). It enables finance, operations, and customer support to manage and audit money flows.

How It Works

Frontend requests a price breakdown from the booking service and then calls the backend to create an internal payment record (status PENDING) via POST /api/payments.

For online payments, backend creates a Razorpay order and returns the order_id and required details to the frontend.

Frontend uses Razorpay Checkout or APIs to collect payment; Razorpay returns payment details and a signature.

Backend verifies signature (webhook or client callback) and then captures/marks payment SUCCESS in the local DB.

For refunds, backend calls Razorpay refund API and updates local payment and booking statuses.

Emails and SMS notifications (Mailtrap/Gmail SMTP for email and Twilio for SMS) are sent on payment success, failure, refund, or chargeback.

External Services Used

Razorpay — payment gateway for India (orders, captures, refunds, webhooks).

Mailtrap (development) / Gmail SMTP (production) — transactional emails (payment receipts, refund confirmation).

Twilio — SMS for OTPs and payment confirmations (optional).

Integration notes: store Razorpay credentials securely (Razorpay Key ID and Key Secret), verify webhook signatures using Razorpay secret, use idempotency keys for order creation, and run in test/sandbox mode during development.

Endpoints

1. Create Payment (internal record and start online flow)

Endpoint: POST /api/payments

Description: Create an internal payment record for a booking. If payment_method is online (e.g., UPI, CARD), backend will optionally create a Razorpay order and return order details needed by frontend.

Request Body (JSON):

```
{
  "payment_reference": "PAY-REF-001",
  "booking_id": 1001,
  "guest_id": 2001,
  "amount": 4500.00,
  "payment_method": "UPI",
  "payment_type": "BOOKING",
  "payment_gateway": "Razorpay",
```

```
"currency": "INR",
"create_razorpay_order": true
}
```

Success Response 201 Created (when internal record created and razorpay order returned):

```
{
"payment_id": 501,
"payment_reference": "PAY-REF-001",
"booking_id": 1001,
"guest_id": 2001,
"amount": 4500.00,
"payment_method": "UPI",
"payment_type": "BOOKING",
"payment_status": "PENDING",
"payment_gateway": "Razorpay",
"currency": "INR",
"razorpay_order": {
"id": "order_Gk1a2B3c4D5eF",
"amount": 450000,
"currency": "INR",
"receipt": "PAY-REF-001",
"status": "created"
},
"created_at": "2025-11-11T08:40:00Z"
}
```

Error Responses:

400 Bad Request — missing fields or invalid amount
409 Conflict — duplicate payment_reference
500 Internal Server Error — gateway or DB issue

Notes:

Razorpay uses smallest currency unit (paise). amount in razorpay_order is in paise (INR 4500 -> 450000).

2. Verify Payment (client callback or webhook verification)

Endpoint: POST /api/payments/verify

Description: Verifies payment signature sent by client after checkout or verifies Razorpay webhook payload and updates local payment_status to SUCCESS or FAILED.

Request Body (from client after checkout):

```
{
  "razorpay_payment_id": "pay_Gk2xYz3aB4",
  "razorpay_order_id": "order_Gk1a2B3c4D5eF",
  "razorpay_signature": "generated_signature",
  "payment_reference": "PAY-REF-001"
}
```

Success Response 200 OK:

```
{
  "payment_id": 501,
  "payment_status": "SUCCESS",
  "transaction_id": "pay_Gk2xYz3aB4",
  "gateway_response": "{...razorpay response...}",
  "captured_at": "2025-11-11T08:42:10Z"
}
```

Error Responses:

401 Unauthorized — signature verification failed

404 Not Found — order or payment reference not found

500 Internal Server Error

Server-side note:

Verify signature using HMAC SHA256 of order_id|payment_id with Razorpay Key Secret.
Only after verification mark payment success and send receipt email and SMS.

3. Capture Payment (for manual capture flow)

Endpoint: POST /api/payments/{payment_id}/capture

Description: Captures an authorized payment (if capture not automatic). Calls Razorpay capture API and updates local record.

Request Body:

```
{
  "amount": 4500.00
}
```

Success Response 200 OK:

```
{
  "payment_id": 501,
  "payment_status": "SUCCESS",
  "captured_at": "2025-11-11T08:45:00Z",
}
```



```
"transaction_id": "pay_Gk2xYz3aB4"
}
```

Error Responses:

400 Bad Request — amount mismatch

409 Conflict — already captured

500 Internal Server Error

Razorpay API used:

POST /v1/payments/{payment_id}/capture

4. Initiate Refund (local + gateway)

Endpoint: POST /api/payments/{payment_id}/refund

Description: Initiates a refund for a payment. Backend calls Razorpay refund API (if gateway payment) and stores refund record.

Request Body:

```
{
  "refund_amount": 4500.00,
  "refund_reason": "Guest cancellation - full refund",
  "refund_method": "original_payment_method"
}
```

Success Response 200 OK:

```
{
  "payment_id": 501,
  "payment_status": "REFUNDED",
  "refund": {
    "refund_id": "rfnd_Gk9xYz4bC",
    "amount": 4500.00,
    "status": "processed",
    "refund_transaction_id": "ref_txn_123",
    "processed_at": "2025-11-11T10:30:00Z"
  }
}
```

Error Responses:

400 Bad Request — amount exceeds original payment

404 Not Found — payment not found

500 Internal Server Error — refund gateway failure

Razorpay API used:

POST /v1/payments/{payment_id}/refund

Notes:

Some gateways charge fees on refunds. Update booking/payment records and send email/SMS to guest.

5. Get All Payments

Endpoint: GET /api/payments

Description: List payments with optional filters (booking_id, guest_id, payment_status, date range, gateway).

Query Example:

GET /api/payments?booking_id=1001&payment_status=SUCCESS&page=1&per_page=20

Success Response 200 OK:

```
[
{
  "payment_id": 501,
  "payment_reference": "PAY-REF-001",
  "booking_id": 1001,
  "guest_id": 2001,
  "amount": 4500.00,
  "payment_method": "UPI",
  "payment_status": "SUCCESS",
  "transaction_id": "pay_Gk2xYz3aB4",
  "payment_gateway": "Razorpay",
  "currency": "INR",
  "payment_date": "2025-11-11T08:42:10Z"
}
```

6. Get Payment by ID

Endpoint: GET /api/payments/{payment_id}

Description: Fetch full details including gateway response and refunds.

Success Response 200 OK:

```
{
  "payment_id": 501,
  "payment_reference": "PAY-REF-001",
  "booking_id": 1001,
  "guest_id": 2001,
```

```
"amount": 4500.00,
"payment_method": "UPI",
"payment_type": "BOOKING",
"payment_status": "SUCCESS",
"transaction_id": "pay_Gk2xYz3aB4",
"payment_gateway": "Razorpay",
"gateway_response": "{...raw gateway JSON...}",
"currency": "INR",
"payment_date": "2025-11-11T08:42:10Z",
"refunds": [
{
"refund_id": "rfnd_Gk9xYz4bC",
"amount": 4500.00,
"status": "processed",
"refund_transaction_id": "ref_txn_123",
"processed_at": "2025-11-11T10:30:00Z"
}
]
}
```

7. Update Payment (status reconciliation)

Endpoint: PUT /api/payments/{payment_id}

Description: Update payment_status, refund details, or gateway info (used for asynchronous updates).

Request Body Example:

```
{
"payment_status": "REFUNDED",
"refund_amount": 4500.00,
"refund_transaction_id": "ref_txn_123",
"refund_date": "2025-11-11T10:30:00Z"
}
```

Success Response 200 OK:

```
{
"message": "Payment record updated successfully."
}
```

8. Delete Payment (not recommended for production)

Endpoint: DELETE /api/payments/{payment_id}

Description: Permanently delete a payment (only for test cleanup or with soft-delete policy).

Success Response 200 OK:

```
{
  "message": "Payment deleted successfully."
}
```

9. Reconcile Payments (batch)

Endpoint: POST /api/payments/reconcile

Description: Runs reconciliation against gateway or internal booking data to detect mismatches.

Request Body Example:

```
{
  "date_from": "2025-11-01",
  "date_to": "2025-11-10",
  "gateway": "Razorpay"
}
```

Success Response:

```
{
  "reconciliation_id": "rec_20251111_01",
  "total_payments": 245,
  "mismatches": 2,
  "generated_at": "2025-11-11T12:00:00Z"
}
```

Usage:

Automate daily or weekly jobs to reconcile payment ledgers with bank/gateway statements.

10. Webhook Endpoint (gateway callbacks)

Endpoint: POST /api/payments/webhook/razorpay

Description: Endpoint to receive Razorpay webhooks for events like payment.captured, payment.failed, refund.processed. Verify signature using X-Razorpay-Signature header and your webhook secret.

Example webhook payload (payment.captured):

```
{
  "entity": "event",
  "event": "payment.captured",
  "payload": {
    "payment": {
      "entity": {
        "id": "pay_Gk2xYz3aB4",
        "order_id": "order_Gk1a2B3c4D5eF",
        "amount": 450000,
        "currency": "INR",
        "status": "captured",
        "method": "upi",
        "created_at": 1700000000
      }
    }
  }
}
```

Response:

200 OK with acknowledgement body.

Security:

Verify HMAC SHA256 signature using webhook secret. Only process events after verification.

Error Codes and Meanings

VALIDATION_ERROR 400 Missing or invalid input

BOOKING_NOT_FOUND 404 Referenced booking does not exist

GUEST_NOT_FOUND 404 Guest not found

PAYMENT_NOT_FOUND 404 Payment record not found

DUPLICATE_REFERENCE 409 Duplicate payment_reference or transaction_id

PAYMENT_FAILED 402 Gateway returned failure

REFUND_FAILED 500 Refund gateway failure

UNAUTHORIZED 401 Signature verification failed or invalid token

INTERNAL_ERROR 500 Unexpected server error

Razorpay Integration Summary (External API details)

Razorpay endpoints (examples, server-to-server)

Create order:

POST <https://api.razorpay.com/v1/orders>

Payload:

```
{
  "amount": 450000,
  "currency": "INR",
  "receipt": "PAY-REF-001",
  "notes": { "booking_id": "1001" },
  "payment_capture": 1
}
```

Response includes order.id which is passed to frontend.

Capture payment (if needed):

POST https://api.razorpay.com/v1/payments/{payment_id}/capture

Refund:

POST https://api.razorpay.com/v1/payments/{payment_id}/refund

Payload:

```
{ "amount": 450000, "speed": "normal", "receipt": "REF-001" }
```

Webhooks:

POST to your </api/payments/webhook/razorpay> with X-Razorpay-Signature header.

Credentials:

RAZORPAY_KEY_ID

RAZORPAY_KEY_SECRET

RAZORPAY_WEBHOOK_SECRET

Notes:

Use Razorpay test mode keys in development. Amounts are in paise. Implement idempotency: if frontend retries, ensure duplicate orders/payments are not created.

Email and SMS Notifications (External)

Email (Mailtrap for dev, Gmail SMTP or production provider for live)

Use to send payment receipts, payment failure alerts, refund confirmations.

Sample Mailtrap payload for payment receipt:

```
{
  "subject": "AaoStays Payment Receipt - PAY-REF-001",
  "to": [{ "email": "john.doe@example.com" }],
  "text": "We have received your payment of ₹4,500 for booking BK-2025-98765-XKJM. Transaction ID: pay_Gk2xYz3aB4."
}
```

SMS (Twilio)

Use for short payment confirmations and OTPs.

Sample Twilio message:

To = +919876543210

From = +12025550123

Body = "AaoStays: Payment of ₹4,500 received for booking BK-2025-98765-XKJM. TXN: pay_Gk2xYz3aB4."

Security and Compliance

Store sensitive keys securely (vault or environment variables).

Do not store full card details on your servers; use gateway tokenization.

Verify webhooks signatures.

Use HTTPS everywhere.

Implement rate-limiting and monitoring.

Keep PCI/DSS considerations in mind when handling card data (prefer gateway-hosted flows or tokenized data).

Audit and Logging

Persist full gateway responses for auditing and dispute resolution.

Log webhook events, signature verification results, and reconciliation runs.

Keep records for regulatory retention (configurable per region).

Sample End-to-End Flow JSON (create payment → verify → capture)

Client requests backend to create payment:

POST /api/payments

Request:

```
{
  "payment_reference": "PAY-REF-001",
  "booking_id": 1001,
  "guest_id": 2001,
  "amount": 4500.00,
  "payment_method": "CARD",
  "payment_type": "BOOKING",
  "payment_gateway": "Razorpay",
  "currency": "INR",
  "create_razorpay_order": true
}
```

Backend response (razorpay order included):

```
{
  "payment_id": 501,
  "payment_reference": "PAY-REF-001",
  "payment_status": "PENDING",
  "razorpay_order": {
    "id": "order_Gk1a2B3c4D5eF",
    "amount": 450000,
    "currency": "INR",
    "receipt": "PAY-REF-001"
  }
}
```

Frontend collects payment using Razorpay Checkout. Razorpay returns payment_id, order_id, signature to frontend.

Frontend posts verification to backend:

POST /api/payments/verify

```
{
  "razorpay_payment_id": "pay_Gk2xYz3aB4",
  "razorpay_order_id": "order_Gk1a2B3c4D5eF",
  "razorpay_signature": "generated_signature",
  "payment_reference": "PAY-REF-001"
}
```

Backend verifies signature and updates:

```
{
  "payment_id": 501,
  "payment_status": "SUCCESS",
  "transaction_id": "pay_Gk2xYz3aB4",
  "gateway_response": "{...}"
}
```

Backend sends receipt email and SMS, and updates booking.payment_status = PAID.

Final Notes (Internal APIs required for production beyond above)

Internal:

/api/payments (CRUD, reconcile) — implemented above

/api/payments/webhook/razorpay — webhook handler (verify signature, update payments)

/api/payments/reconcile — batch reconciliation jobs

/api/payments/report — finance reports and exports (CSV/PDF)

External (already listed):

Razorpay — orders, captures, refunds, webhooks

Mailtrap / Gmail SMTP — transactional emails

Twilio — SMS notifications

REVIEW API DOCUMENTATION

Base URL:

Authentication:

Bearer Token required

Authorization: Bearer {your_access_token}

Overview

The Review API manages ratings and feedback submitted by guests after completing a booking.

Each review is linked to a booking, property, and guest.

It supports creating reviews, retrieving them, updating content or visibility, and deleting or moderating user feedback.

This API helps maintain trust and transparency for future guests.

Why This API Is Needed

Reviews are crucial for building credibility, improving listings, and increasing conversions. They help future guests evaluate properties, provide hosts insights for improvement, and give admins tools for moderation.

The Review API ensures structured, reliable collection and display of feedback.

How It Works

A guest finishes a stay and becomes eligible to review the booking.

They submit ratings, title, review text, and optional category ratings.

The review is stored with default flags (is_visible = true, is_verified = false).

Admins can verify, feature, or hide reviews.

Other users may mark reviews as helpful.

Each booking can have only one review to avoid spam and duplicate influence.

1. Create a Review

Endpoint:

POST /api/reviews

Description:

Allows a guest to submit a new review for a completed booking.

A booking can have only one review (unique constraint on booking_id).

Request Body (JSON):

```
{
  "booking_id": 1001,
  "property_id": 301,
  "guest_id": 2001,
  "overall_rating": 4.8,
  "cleanliness_rating": 5.0,
  "accuracy_rating": 4.5,
  "communication_rating": 4.8,
  "location_rating": 4.6,
  "value_rating": 4.7,
  "check_in_rating": 4.9,
  "review_title": "Fantastic stay!",
  "review_text": "The apartment was spotless, and the host was very responsive. Highly recommended!"
}
```

Success Response (201 Created):

```
{
  "review_id": 501,
  "booking_id": 1001,
  "property_id": 301,
  "guest_id": 2001,
  "overall_rating": 4.8,
  "cleanliness_rating": 5.0,
  "accuracy_rating": 4.5,
  "communication_rating": 4.8,
}
```

```
"location_rating": 4.6,  
"value_rating": 4.7,  
"check_in_rating": 4.9,  
"review_title": "Fantastic stay!",  
"review_text": "The apartment was spotless, and the host was very responsive. Highly  
recommended!",  
"is_verified": false,  
"is_visible": true,  
"is_featured": false,  
"helpful_count": 0,  
"created_at": "2025-11-11T10:15:00Z"  
}
```

2. Retrieve All Reviews

Endpoint:

GET /api/reviews

Description:

Fetches all reviews in the system.

Supports multiple filters for properties, guests, visibility, and featured reviews.

Query Parameters:

property_id

guest_id

is_visible

is_featured

Example Request:

GET /api/reviews?property_id=301&is_visible=true

Example Response (200 OK):

```
[  
{  
  "review_id": 501,  
  "property_id": 301,  
  "guest_id": 2001,  
  "overall_rating": 4.8,  
  "review_title": "Fantastic stay!",  
  "review_text": "The apartment was spotless, and the host was very responsive.",  
  "is_featured": false,  
  "created_at": "2025-11-11T10:15:00Z"  
},  
{
```

```
"review_id": 502,  
"property_id": 301,  
"guest_id": 2002,  
"overall_rating": 4.2,  
"review_title": "Good location",  
"review_text": "Nice apartment, close to public transport. Could be cleaner.",  
"is_featured": true,  
"created_at": "2025-11-09T09:30:00Z"  
}  
]
```

3. Retrieve a Single Review

Endpoint:

GET /api/reviews/{review_id}

Description:

Fetches full details of a single review along with host response.

Example Request:

GET /api/reviews/501

Success Response (200 OK):

```
{  
  "review_id": 501,  
  "booking_id": 1001,  
  "property_id": 301,  
  "guest_id": 2001,  
  "overall_rating": 4.8,  
  "cleanliness_rating": 5.0,  
  "accuracy_rating": 4.5,  
  "communication_rating": 4.8,  
  "location_rating": 4.6,  
  "value_rating": 4.7,  
  "check_in_rating": 4.9,  
  "review_title": "Fantastic stay!",  
  "review_text": "The apartment was spotless, and the host was very responsive.",  
  "host_response": "Thank you for your kind words!",  
  "host_response_date": "2025-11-12T08:00:00Z",  
  "is_verified": true,  
  "is_visible": true,  
  "is_featured": false,  
  "helpful_count": 12,
```

```
"created_at": "2025-11-11T10:15:00Z"
}
```

4. Update a Review

Endpoint:

PUT /api/reviews/{review_id}

Description:

Allows updating review text, ratings, visibility flags, or adding a host response.

Example Request (Guest Updating):

```
{
  "review_text": "Great stay overall. The location was perfect!",
  "overall_rating": 4.7,
  "is_visible": true
}
```

Example Request (Host Response):

```
{
  "host_response": "Thank you for staying with us!",
  "host_response_date": "2025-11-12T08:00:00Z"
}
```

Response (200 OK):

```
{
  "message": "Review updated successfully."
}
```

5. Delete a Review

Endpoint:

DELETE /api/reviews/{review_id}

Description:

Removes a review permanently from the system.

Used for administrative moderation or when content violates guidelines.

Success Response:

```
{
  "message": "Review deleted successfully."
}
```

6. Mark Review as Helpful

Endpoint:

POST /api/reviews/{review_id}/helpful

Description:

Increments helpful_count for a review when another user marks it useful.

Example Response:

```
{  
  "message": "Marked as helpful.",  
  "helpful_count": 13  
}
```

Error Responses

VALIDATION_ERROR — 400

BOOKING_NOT_FOUND — 404

REVIEW_EXISTS — 409

REVIEW_NOT_FOUND — 404

UNAUTHORIZED_ACTION — 403

INTERNAL_ERROR — 500

Database Relationship

Each review belongs to:

one booking

one property

one guest

Deleting a booking/property/guest cascades reviews.

Each booking can have only one review (unique constraint on booking_id).

Example Use Cases

Guest completes stay → submits rating and comments

Host responds to feedback

Admin verifies or features a high-quality review

Other guests mark reviews as helpful to increase credibility

WISHLIST API DOCUMENTATION

Base URL:

Authentication:

Bearer Token required

Authorization: Bearer {your_access_token}

Overview

The Wishlist API allows guests to save properties they are interested in.

It helps users revisit properties easily, improves engagement, and increases conversion rates.

Each wishlist entry links a guest to a property and can include personal notes.

Why This API Is Needed

Guests often browse multiple properties and want to save favorites for later.

Wishlists improve user retention, encourage return visits, and serve as a key personalization feature in booking platforms like Airbnb, OYO, and Booking.com.

How It Works

A guest clicks “Add to Wishlist” for a property.

The system creates a unique wishlist record linking guest and property.

Guests can fetch, update notes, or remove items from their wishlist.

A unique constraint prevents the same property from being added twice by the same guest.

Deleting the guest or property also deletes related wishlist items automatically.

1. Add Property to Wishlist

Endpoint:

POST /api/wishlists

Description:

Adds a property to a guest’s wishlist.

If the item already exists for the guest, a conflict error is returned.

Request Body (JSON):

```
{
  "guest_id": 3001,
  "property_id": 501,
  "notes": "Looks perfect for summer vacation."
}
```

Success Response (201 Created):

```
{
  "wishlist_id": 1001,
  "guest_id": 3001,
  "property_id": 501,
  "notes": "Looks perfect for summer vacation.",
  "added_at": "2025-11-11T10:15:00Z"
}
```

2. Retrieve Guest Wishlist

Endpoint:

GET /api/wishlists?guest_id={guest_id}

Description:

Fetches all wishlist items for a specific guest.

Supports optional filters and sorting.

Example Request:

GET /api/wishlists?guest_id=3001

Success Response (200 OK):

```
[
  {
    "wishlist_id": 1001,
    "guest_id": 3001,
    "property_id": 501,
    "notes": "Looks perfect for summer vacation.",
    "added_at": "2025-11-11T10:15:00Z"
  },
  {
    "wishlist_id": 1002,
    "guest_id": 3001,
    "property_id": 502,
    "notes": "Great for a quick weekend trip.",
    "added_at": "2025-11-10T09:00:00Z"
  }
]
```



```
}  
]
```

3. Retrieve a Single Wishlist Item

Endpoint:

GET /api/wishlists/{wishlist_id}

Description:

Returns detailed information of a specific wishlist entry.

Example Request:

GET /api/wishlists/1001

Success Response (200 OK):

```
{  
  "wishlist_id": 1001,  
  "guest_id": 3001,  
  "property_id": 501,  
  "notes": "Looks perfect for summer vacation.",  
  "added_at": "2025-11-11T10:15:00Z"  
}
```

4. Update Wishlist Notes

Endpoint:

PUT /api/wishlists/{wishlist_id}

Description:

Updates personal notes for a wishlist property.

Request Body:

```
{  
  "notes": "Planning to book this for Christmas vacation."  
}
```

Success Response (200 OK):

```
{  
  "message": "Wishlist notes updated successfully."  
}
```

5. Remove Property From Wishlist

Endpoint:

DELETE /api/wishlists/{wishlist_id}

Description:

Deletes a wishlist entry permanently.

Success Response:

```
{  
  "message": "Property removed from wishlist successfully."  
}
```

Error Responses

VALIDATION_ERROR — 400

WISHLIST_EXISTS — 409

WISHLIST_NOT_FOUND — 404

GUEST_NOT_FOUND — 404

PROPERTY_NOT_FOUND — 404

UNAUTHORIZED_ACTION — 403

INTERNAL_ERROR — 500

Database Relationship

Each wishlist record connects:

one guest

one property

Deleting a guest or property removes associated wishlist items (ON DELETE CASCADE).

Each guest-property pair is unique.

Example Use Cases

Guest browsing properties → adds interesting ones to wishlist

Guest planning a trip later → revisits wishlist to choose a property

Guest adds notes to remember why they saved a property

Guest removes properties they are no longer interested in

